

Introduction to Graphs

- What is a Graph?
 - (A collection of nodes (vertices) connected by edges.)
- Real-world Examples:
 - Google Maps (Cities & Roads)
 - Social Networks (People & Friendships)
 - Internet (Webpages & Links)

Basic Terminology

1. Vertex (Node)
2. Edge (Link between two nodes)
3. Adjacent vertices
4. Degree (Indegree & Outdegree)
5. Path, Cycle, Connected Graph
6. Directed vs Undirected Graphs
7. Weighted vs Unweighted Graphs

Graph Representation

- Adjacency Matrix
 - (2D array, Good for dense graphs)
- Adjacency List
 - (List of neighbors, Good for sparse graphs)



Simple Example:

<https://www.geeksforgeeks.org/graph-and-its-representations/>

Types of Graphs

- Directed vs Undirected
- Weighted vs Unweighted
- Cyclic vs Acyclic
- Connected vs Disconnected
- Special: Trees, DAGs (Directed Acyclic Graphs)

Graph Traversal

- Breadth-First Search (BFS)
 - Use Queue
 - Level-order traversal
 - Applications: Shortest Path, Social Network Analysis

- Depth-First Search (DFS)
 - Use Stack (recursion or explicit stack)
 - Go as deep as possible
 - Applications: Cycle Detection, Topological Sort

Common Graph Algorithms

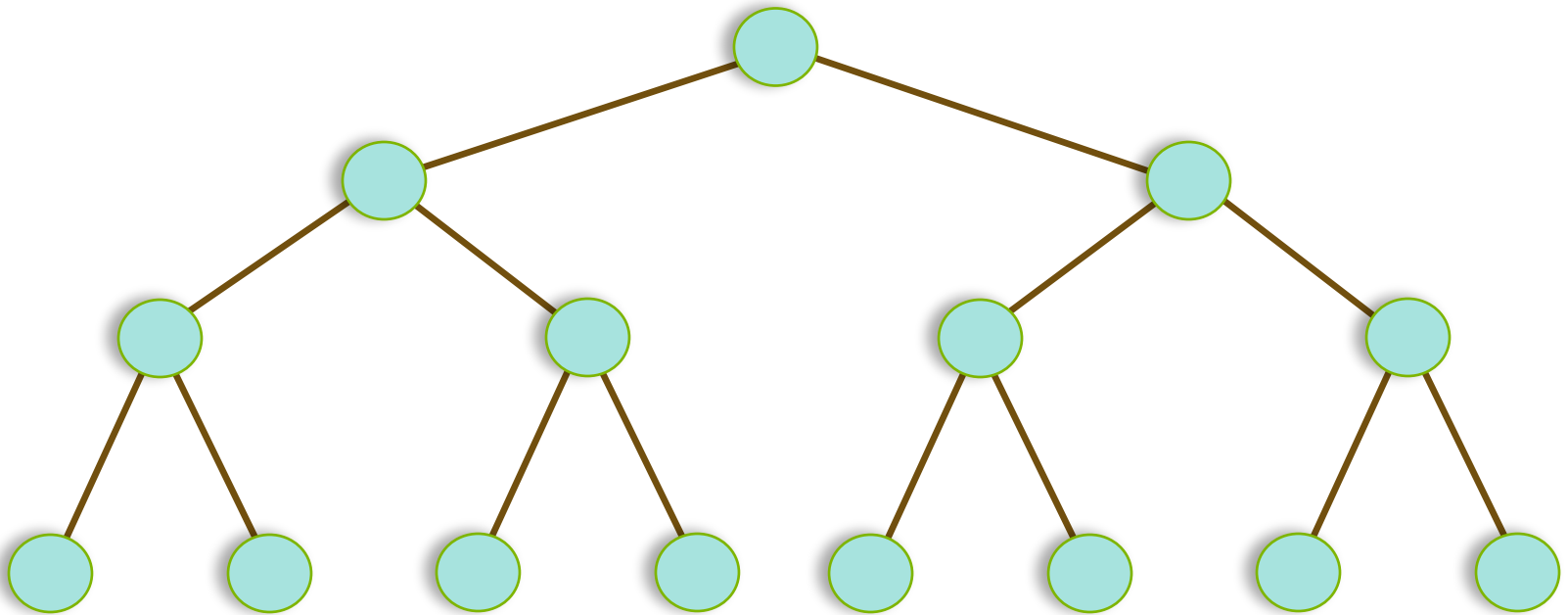
- Shortest Path Algorithms
 - Dijkstra's Algorithm (Weighted, Non-negative)
 - Bellman-Ford (Handles Negative weights)
- Minimum Spanning Tree
 - Prim's Algorithm
 - Kruskal's Algorithm
- Topological Sorting
 - For Directed Acyclic Graphs (DAGs)
- Cycle Detection
 - Using DFS

Applications of Graphs

- Routing and Navigation (GPS)
- Social Networks Analysis
- Scheduling (Task dependencies)
- Network Flow (Internet packet routing)
- AI (State-space graphs in search problems)
- Web Page Ranking (Google's PageRank)

Graph Searching Algorithms

Tree



Breadth-First-Search(BFS)

Definition:

BFS is a graph traversal algorithm that explores all neighbors at the current depth before moving to nodes at the next depth level.

□ Key Point: Level by Level traversal.

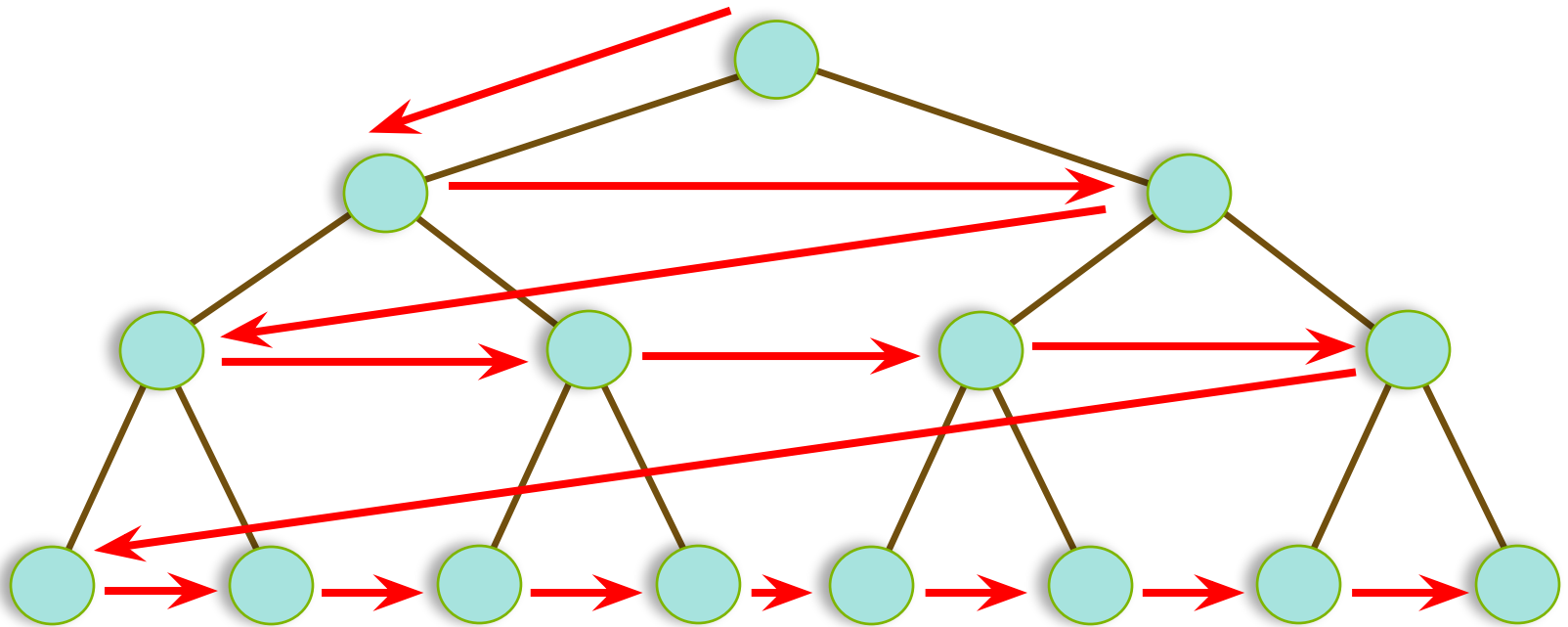
BFS Pseudocode



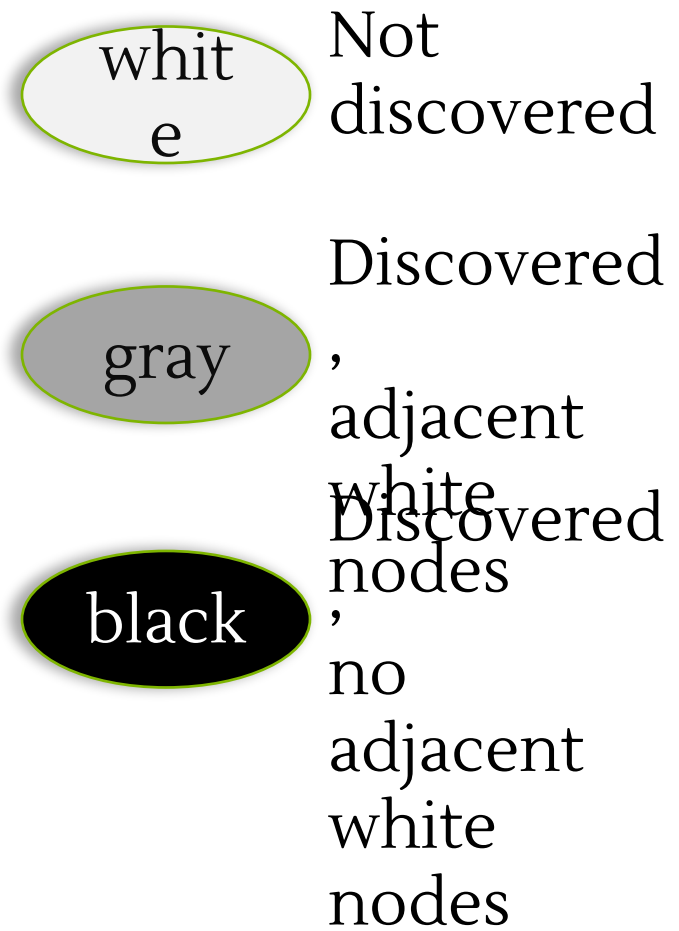
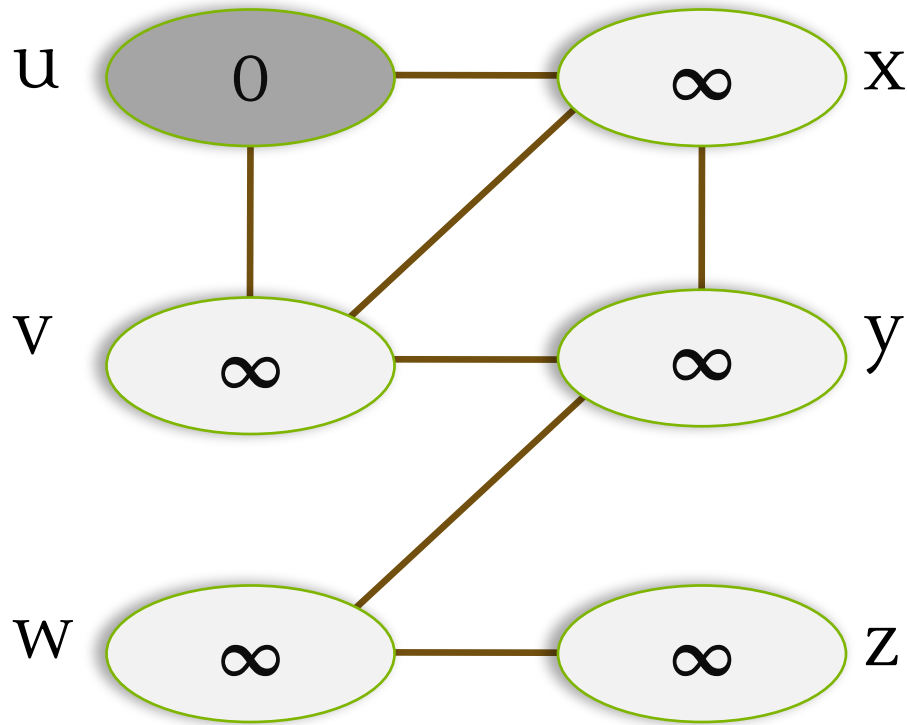
BFS(Graph, StartNode):

1. Initialize a queue to keep track of nodes to visit.
2. Mark the StartNode as visited and enqueue it.
3. While the queue is not empty:
 - Dequeue a node from the front of the queue (currentNode).
 - Process the currentNode (this could mean printing the node, checking a condition, etc.).
 - For each neighbor of currentNode:
 - If the neighbor has not been visited:
 - Mark it as visited.
 - Enqueue the neighbor.

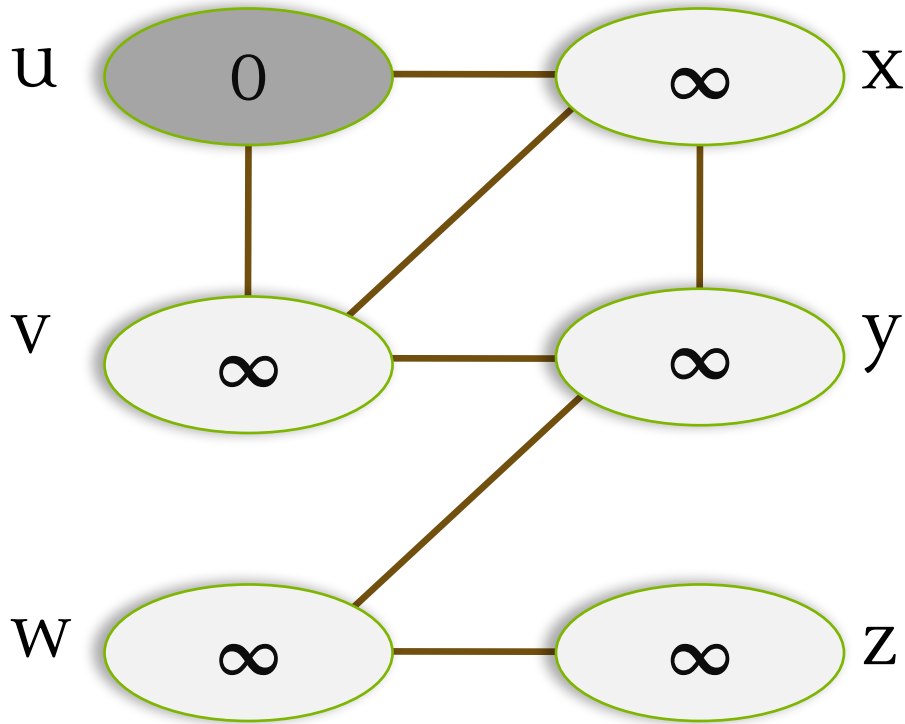
Breadth-First Search (BFS)



Breadth-First Search (BFS)



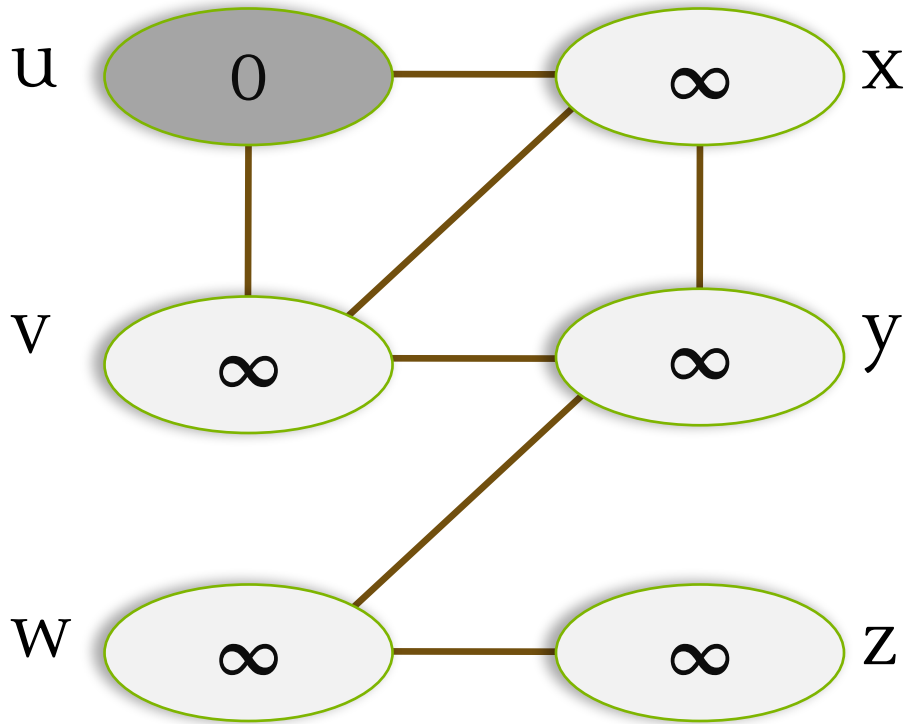
Breadth-First Search (BFS)



BFS(G, u):

1. Initialize the graph
 $\text{color}[u] \leftarrow \text{gray}$
 $\pi[u] \leftarrow \text{Nil}$
 $d[u] \leftarrow 0$
 for each other
 vertex
 $\text{color}[u] \leftarrow \text{white}$

Breadth-First Search (BFS)



Q u

BFS(G, u):

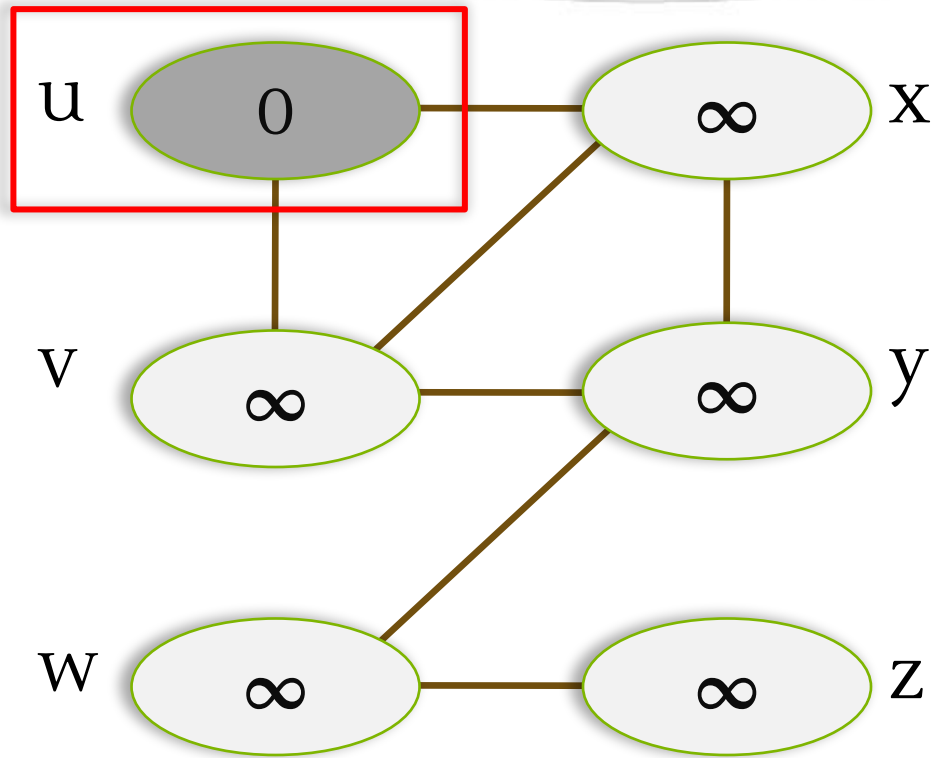
2. Initialize the queue

$Q \leftarrow \emptyset$

Enqueue(Q, u)

Breadth-First Search (BFS)

$t = u$



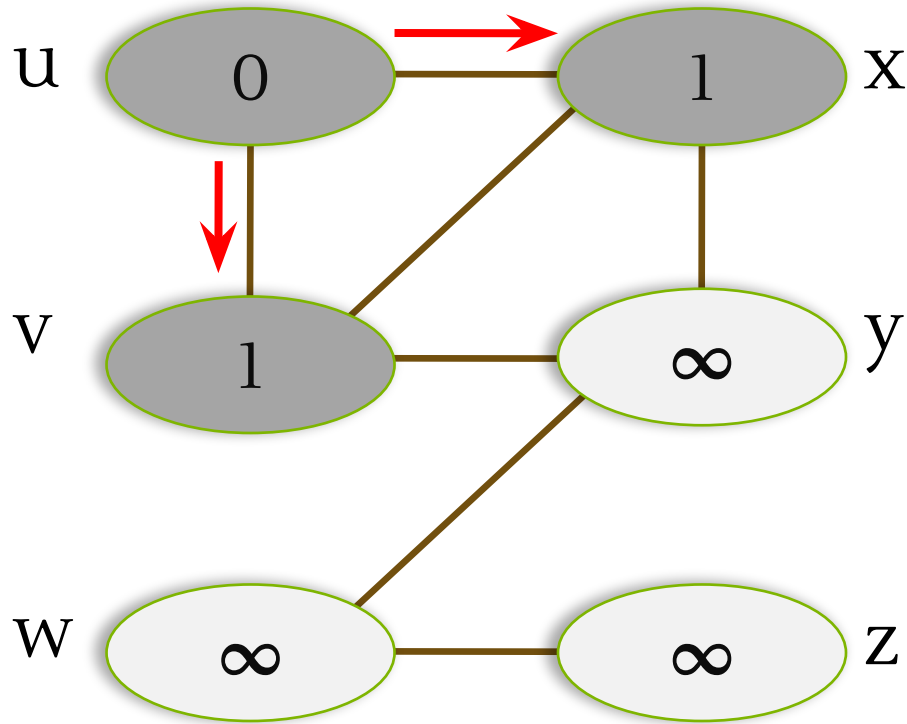
Q

BFS(G, u):

3. While $Q \neq \emptyset$

1) $t \leftarrow \text{Dequeue}(Q)$

Breadth-First Search (BFS)



$t = u$

$r = x, v$

Q

v	x
---	---

BFS(G, u):

3. While $Q \neq \emptyset$

2) for each r adj to t

if $\text{color}[r] = \text{white}$

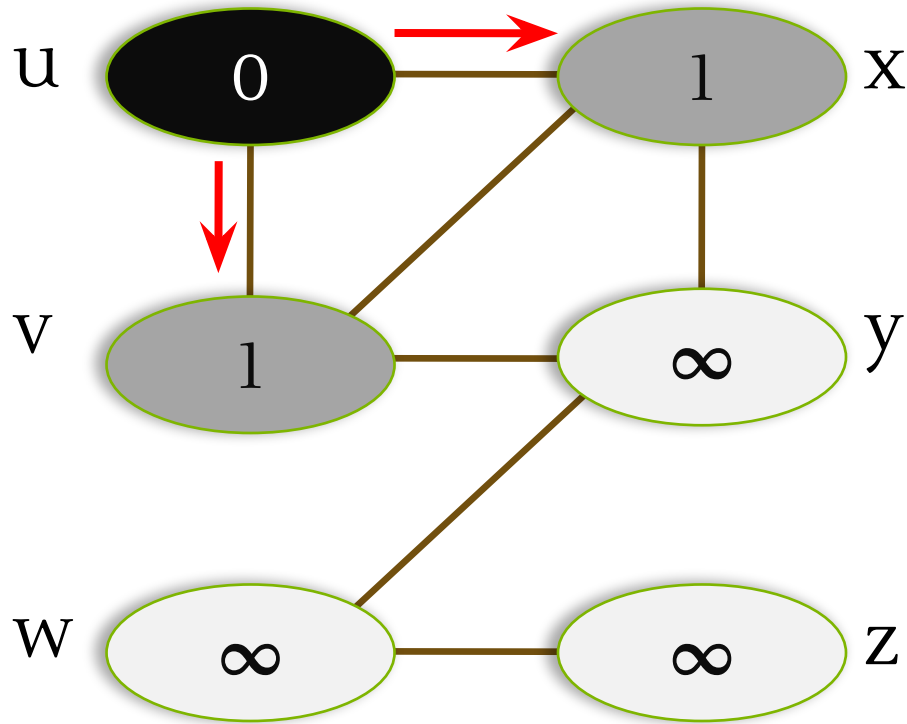
$\text{color}[r] \leftarrow \text{gray}$

$\pi[r] \leftarrow t$

$d[r] \leftarrow d[t] + 1$

Enqueue(Q, r)

Breadth-First Search (BFS)



$t = u$
 $r = x, v$

Q

v	x
---	---

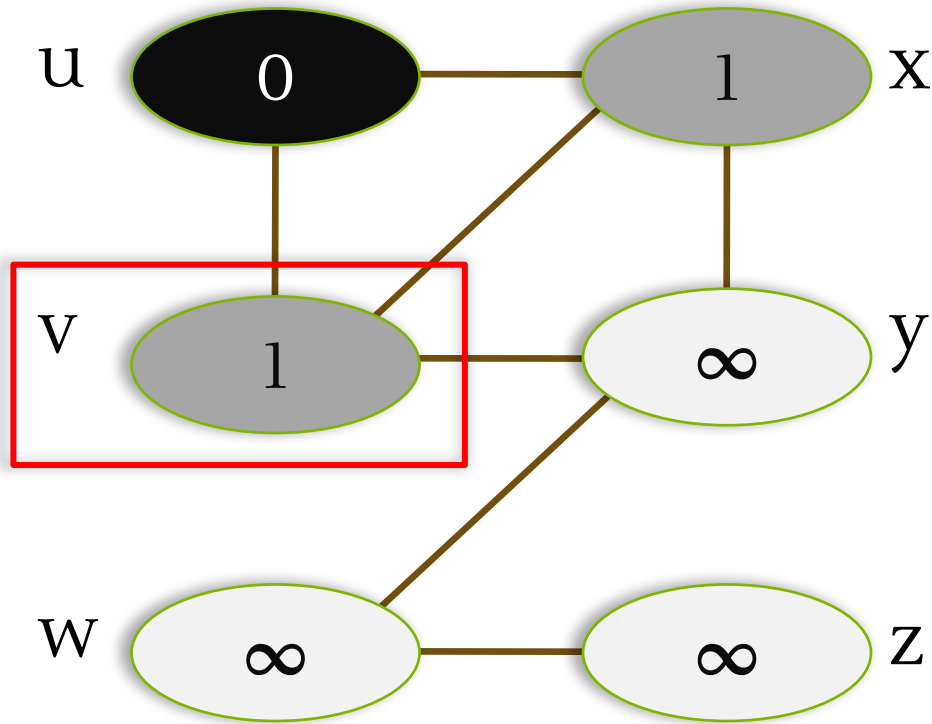
BFS(G, u):

3. While $Q \neq \emptyset$

3) $\text{color}[t] \leftarrow \text{black}$

Breadth-First Search (BFS)

$t = v$



Q x

BFS(G, u):

3. While $Q \neq \emptyset$

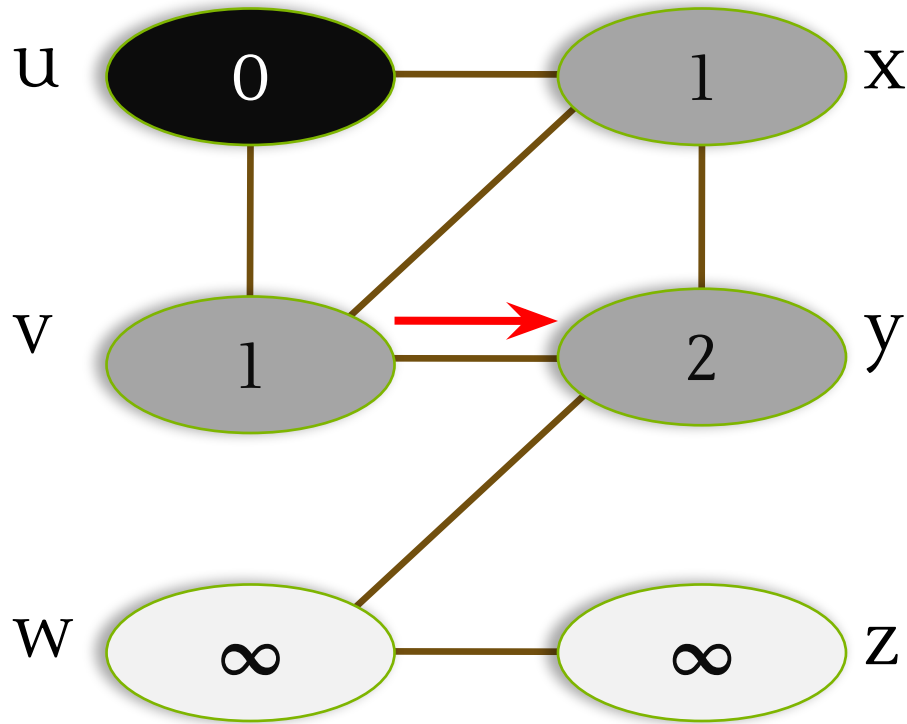
1) $t \leftarrow \text{Dequeue}(Q)$

2) for each r adj to t

...

3) $\text{color}[t] \leftarrow \text{black}$

Breadth-First Search (BFS)



$t = v$
 $r = y$
Q

x	y
---	---

BFS(G, u):

3. While $Q \neq \emptyset$

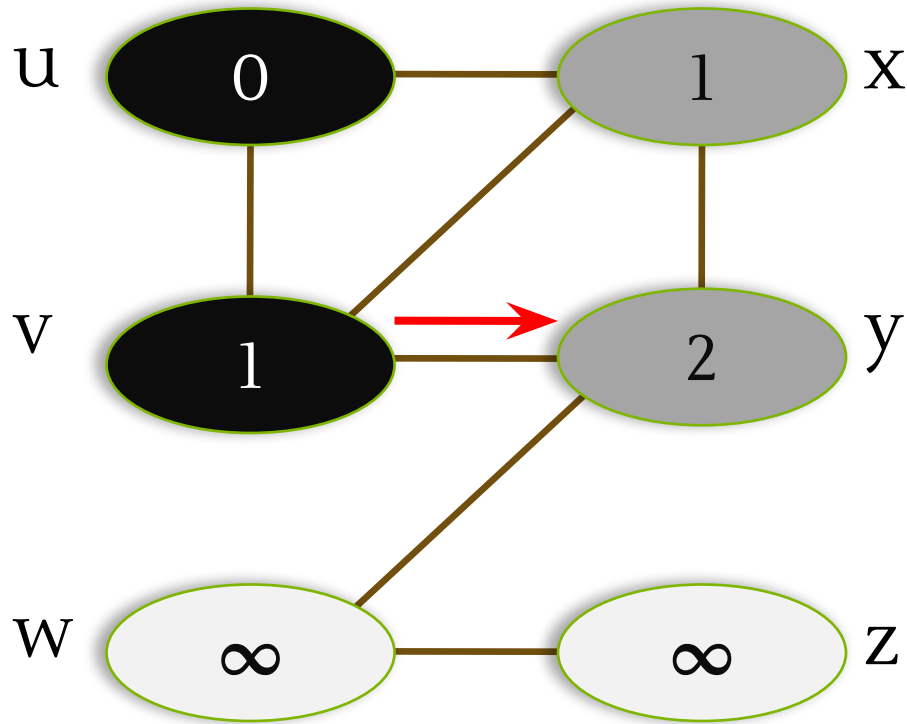
1) $t \leftarrow \text{Dequeue}(Q)$

2) for each r adj to t

...

3) $\text{color}[t] \leftarrow \text{black}$

Breadth-First Search (BFS)



$t = v$
 $r = y$
 Q

x	y
-----	-----

BFS(G, u):

3. While $Q \neq \emptyset$

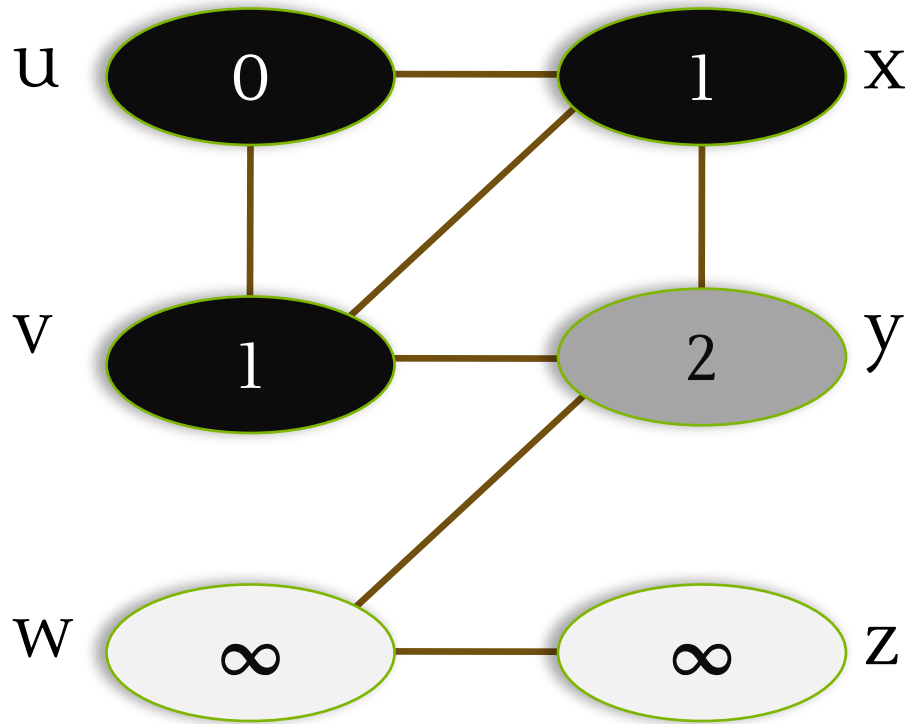
1) $t \leftarrow \text{Dequeue}(Q)$

2) for each r adj to t

...

3) $\text{color}[t] \leftarrow \text{black}$

Breadth-First Search (BFS)



$t = x$

$r =$

Q y

BFS(G, u):

3. While $Q \neq \emptyset$

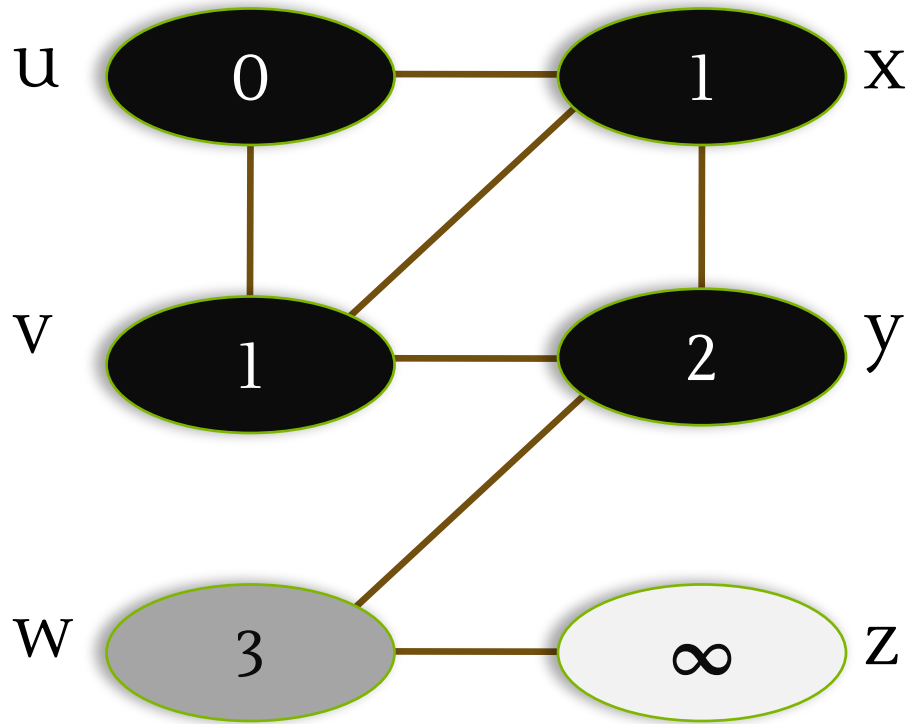
1) $t \leftarrow \text{Dequeue}(Q)$

2) for each r adj to t

...

3) $\text{color}[t] \leftarrow \text{black}$

Breadth-First Search (BFS)



$t = y$
 $r = w$

Q w

BFS(G, u):

3. While $Q \neq \emptyset$

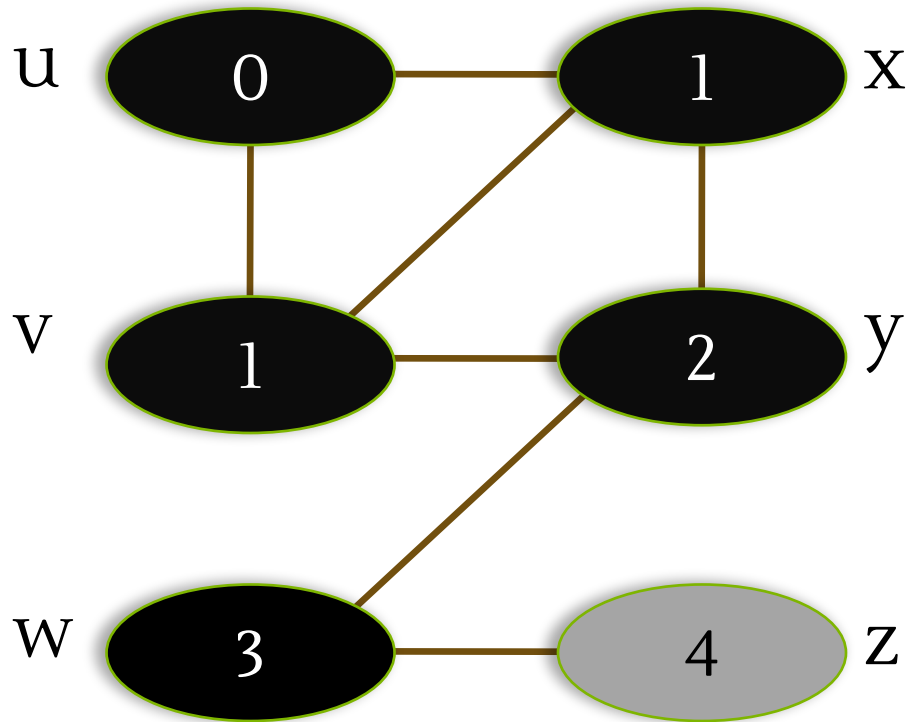
1) $t \leftarrow \text{Dequeue}(Q)$

2) for each r adj to t

...

3) $\text{color}[t] \leftarrow \text{black}$

Breadth-First Search (BFS)



$t = w$

$r = z$

Q z

BFS(G, u):

3. While $Q \neq \emptyset$

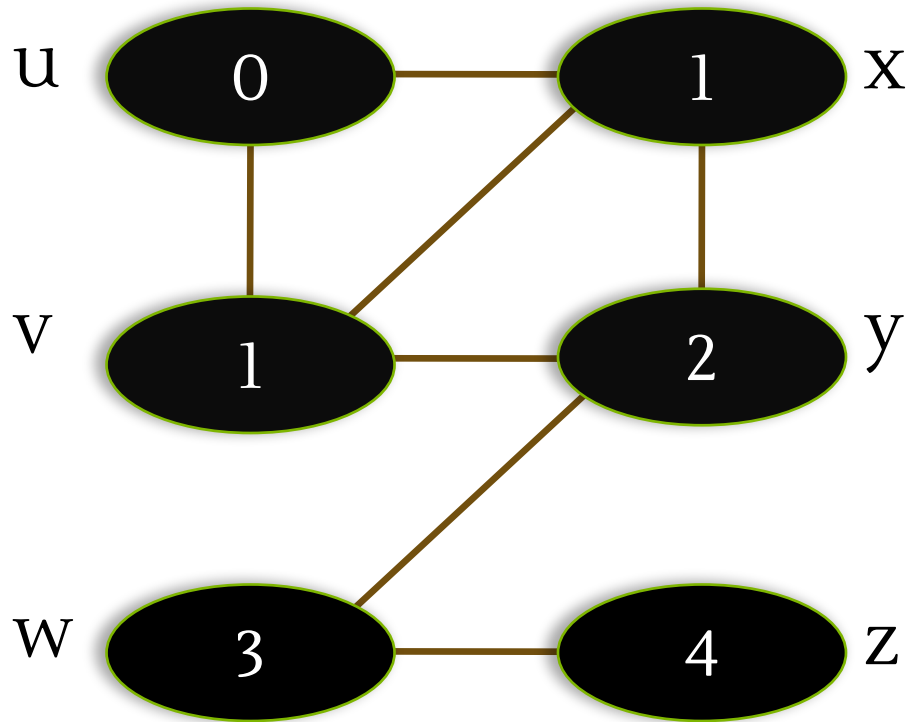
1) $t \leftarrow \text{Dequeue}(Q)$

2) for each r adj to t

...

3) $\text{color}[t] \leftarrow \text{black}$

Breadth-First Search (BFS)



Q

$t = z$

$r =$

BFS(G, u):

3. While $Q \neq \emptyset$

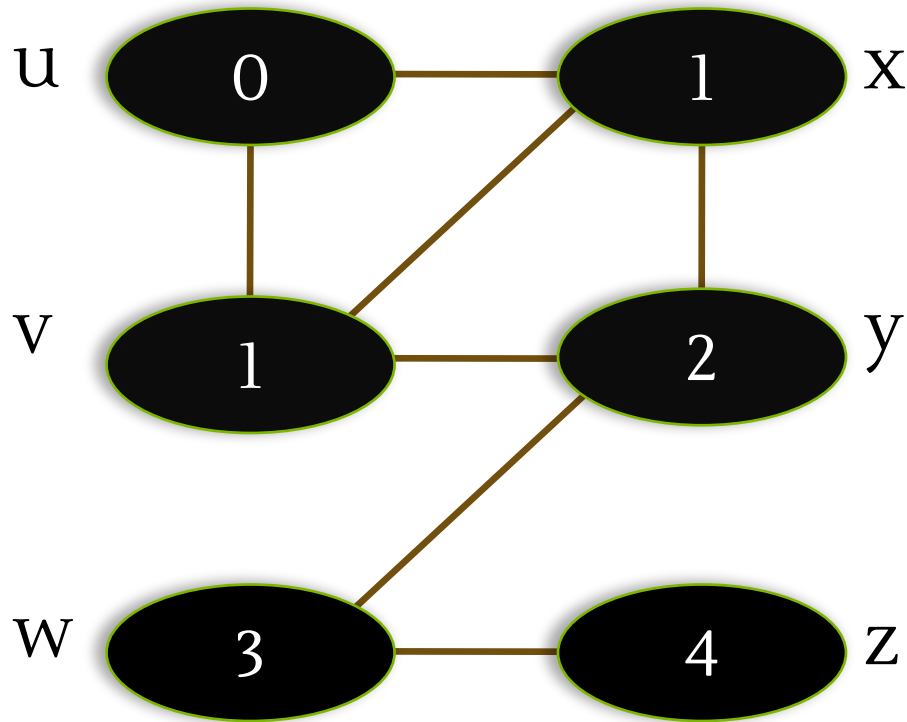
1) $t \leftarrow \text{Dequeue}(Q)$

2) for each r adj to t

...

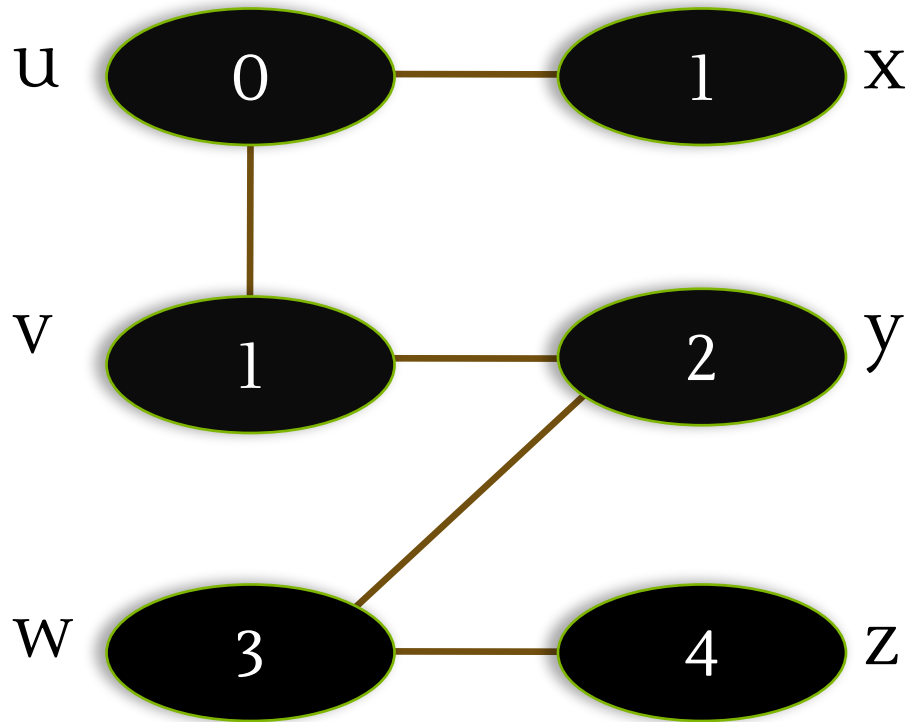
3) $\text{color}[t] \leftarrow \text{black}$

Breadth-First Search (BFS)



BFS(G, u):
- the shortest-path
distance
from u

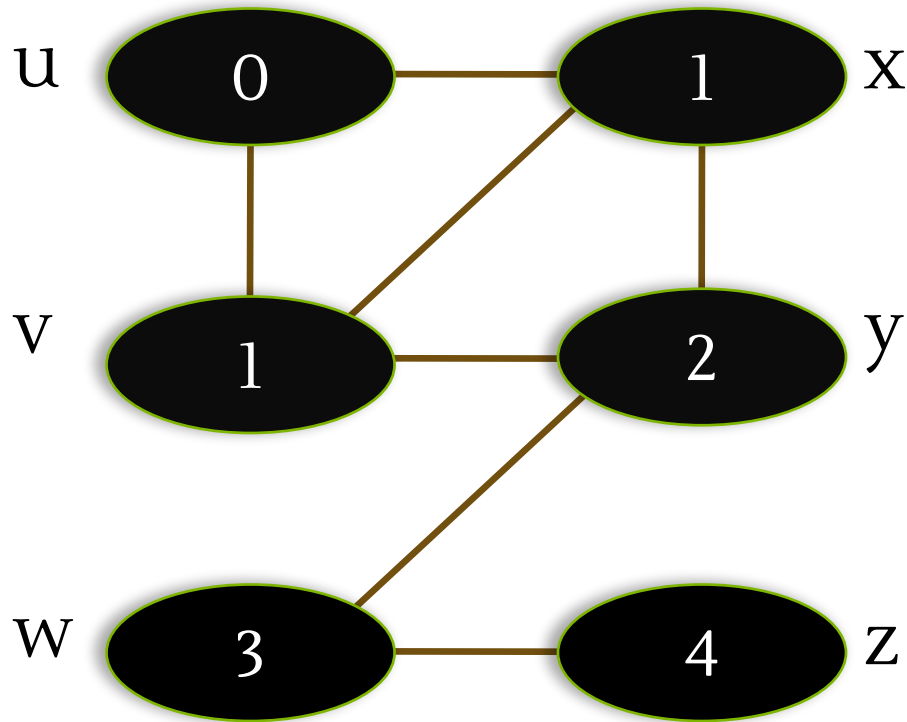
Breadth-First Search (BFS)



BFS(G, u):

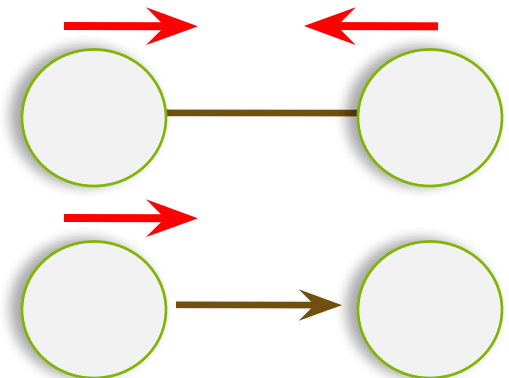
- the shortest-path distance from u
- construct a tree

Breadth-First Search (BFS)

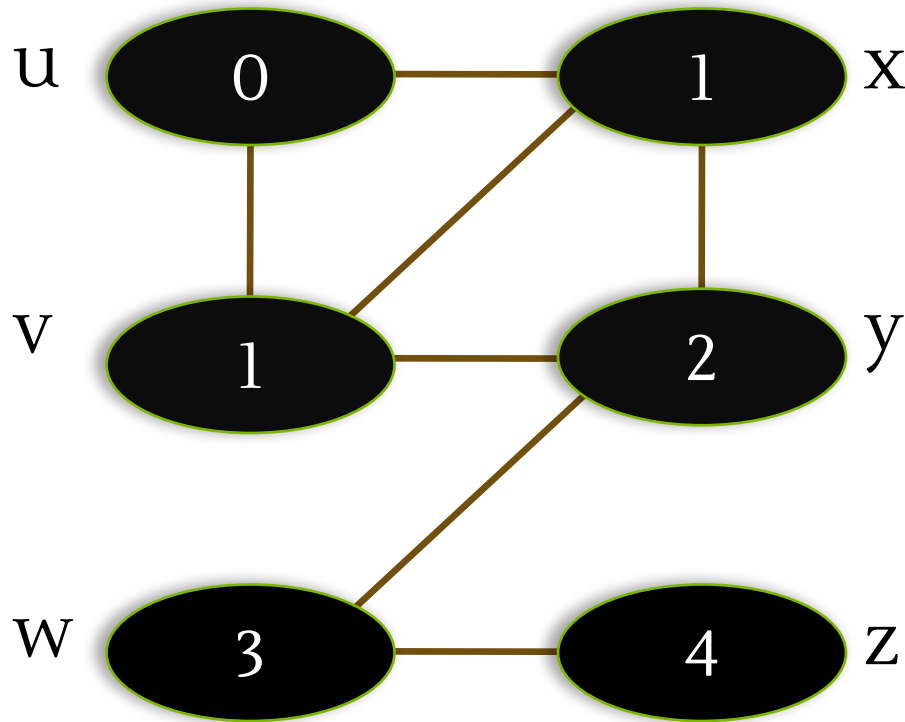


BFS(G, u):

- Initialization: $|V|$
- Enqueueing/dequeueing: $|V|$
- Scanning adj vertices: $|E|$



Breadth-First Search (BFS)



BFS(G, u):

- Initialization: $O(|V|)$
 - Enqueueing/dequeueing: $O(|V|)$
 - Scanning adjacent vertices: $O(|E|)$
- => total running time: $O(|V| + |E|)$

Time & Space Complexity (BFS)

Aspect	Complexity
Time Complexity	$O(V + E)$
Space Complexity	$O(V)$ (or $O(V + E)$ if including the graph)

Breadth-First Search (BFS)

Applications

1. Shortest Path in Unweighted Graphs
 - BFS finds the shortest path from the source to all other nodes (because it visits level by level).
 - Used in navigation systems like Google Maps when all roads are considered equal.
2. Web Crawlers
 - BFS helps in crawling websites layer by layer (first all pages linked from the homepage, then their links, and so on).
3. Social Networking Sites
 - Finding people within k degrees of connection (e.g., friend suggestions on Facebook).
4. Broadcast Networks
 - In networking, when a broadcast message is sent from one node to all other nodes in the least number of hops.

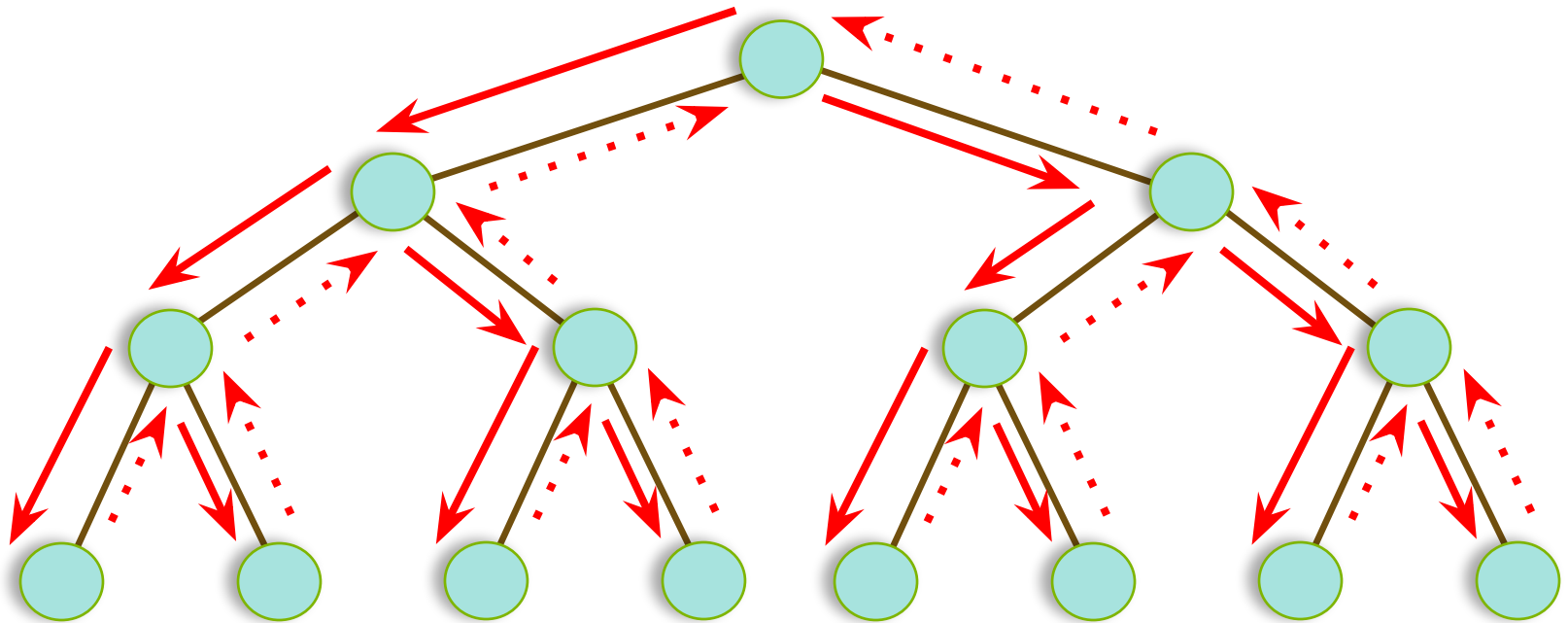
Depth-First Search (DFS)

Definition:

DFS is a graph traversal algorithm that explores as far along a branch as possible before backtracking.

Key Point: Depth before Breadth (opposite of BFS).

Depth-First Search (DFS)



DFS Flow (Recursive Pseudocode)

function DFS(Graph, currentNode, Visited):

- mark currentNode as visited

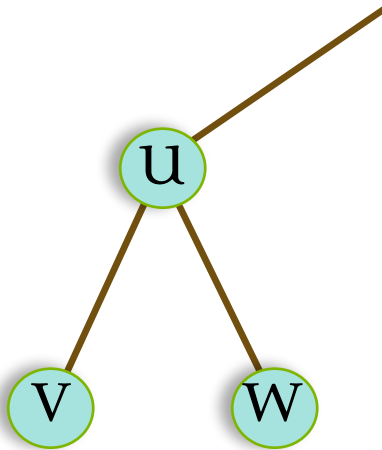
- process currentNode

- for each neighbor of currentNode:

 - if neighbor is not visited:

 - DFS(Graph, neighbor, Visited)

Depth-First Search (DFS)

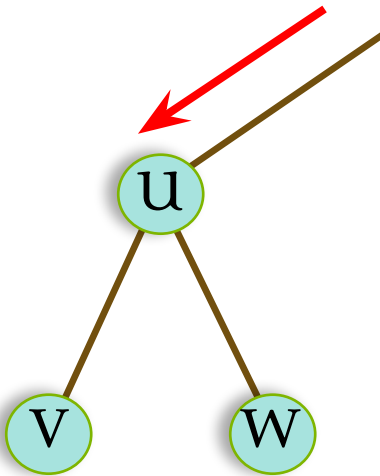


$d[u]$: when u is discovered
 $f[u]$: when searching adj of u
is finished

Depth-First Search (DFS)

timestamp: t

$d[u] = t$

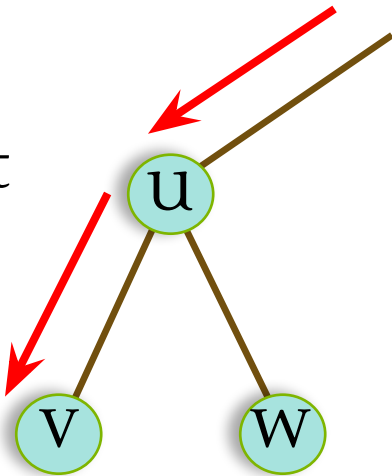


$d[u]$: when u is discovered
 $f[u]$: when searching adj of u
is finished

Depth-First Search (DFS)

timestamp: $t+1$

$d[u] = t$



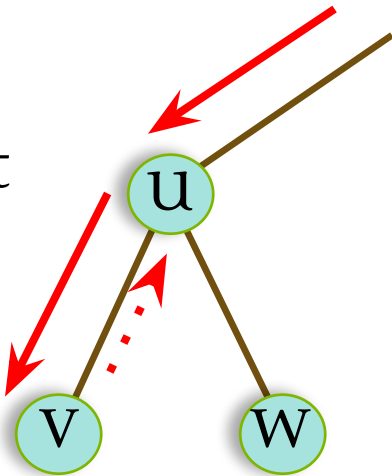
$d[v] = t+1$

$d[u]$: when u is discovered
 $f[u]$: when searching adj of u is finished

Depth-First Search (DFS)

timestamp: $t+2$

$d[u] = t$



$d[v] =$

$t+1$

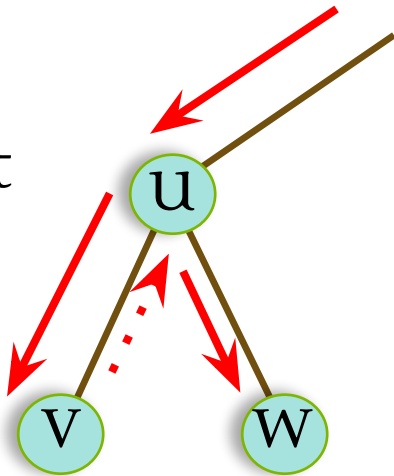
$f[v] = t+2$

$d[u]$: when u is discovered
 $f[u]$: when searching adj of u
is finished

Depth-First Search (DFS)

timestamp: $t+3$

$d[u] = t$



$d[v] =$

$t+1$

$f[v] = t+2$

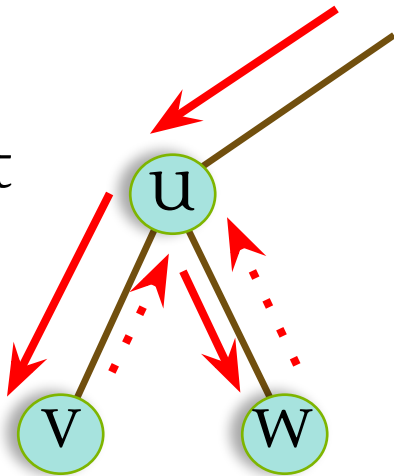
$d[w] = t+3$

$d[u]$: when u is discovered
 $f[u]$: when searching adj of u
is finished

Depth-First Search (DFS)

timestamp: $t+4$

$d[u] = t$



$d[v] =$
 $t+1$

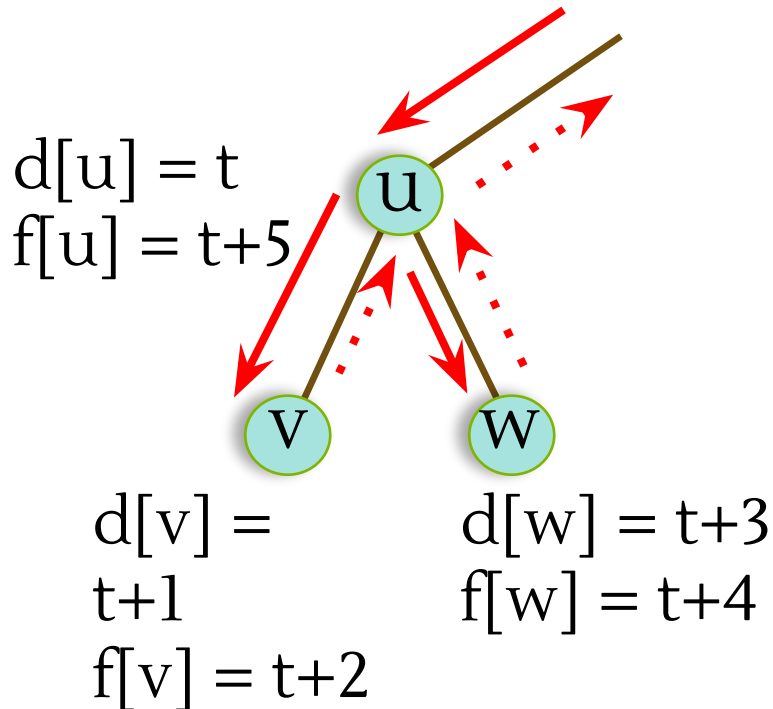
$f[v] = t+2$

$d[w] = t+3$
 $f[w] = t+4$

$d[u]$: when u is discovered
 $f[u]$: when searching adj of u
is finished

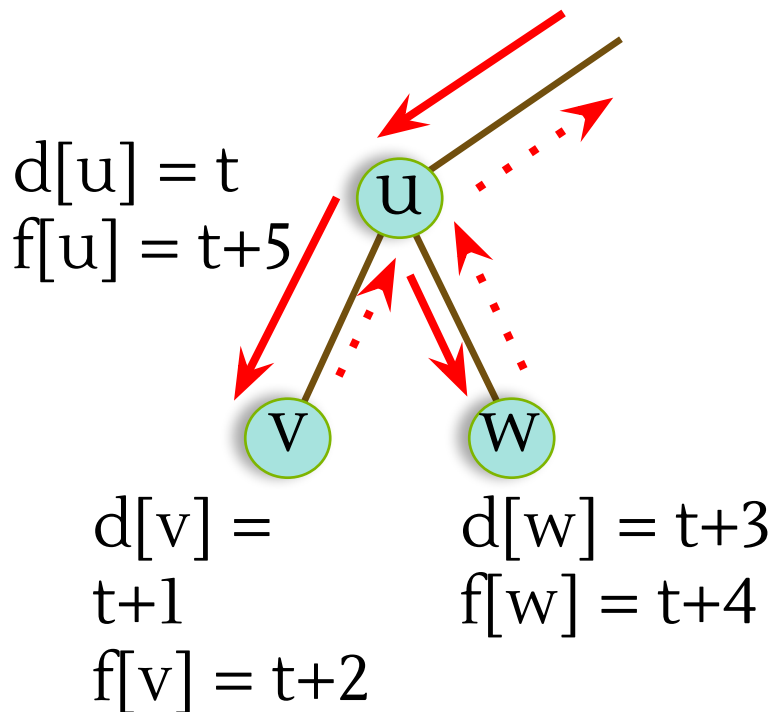
Depth-First Search (DFS)

timestamp: $t+5$



$d[u]$: when u is discovered
 $f[u]$: when searching adj of u is finished

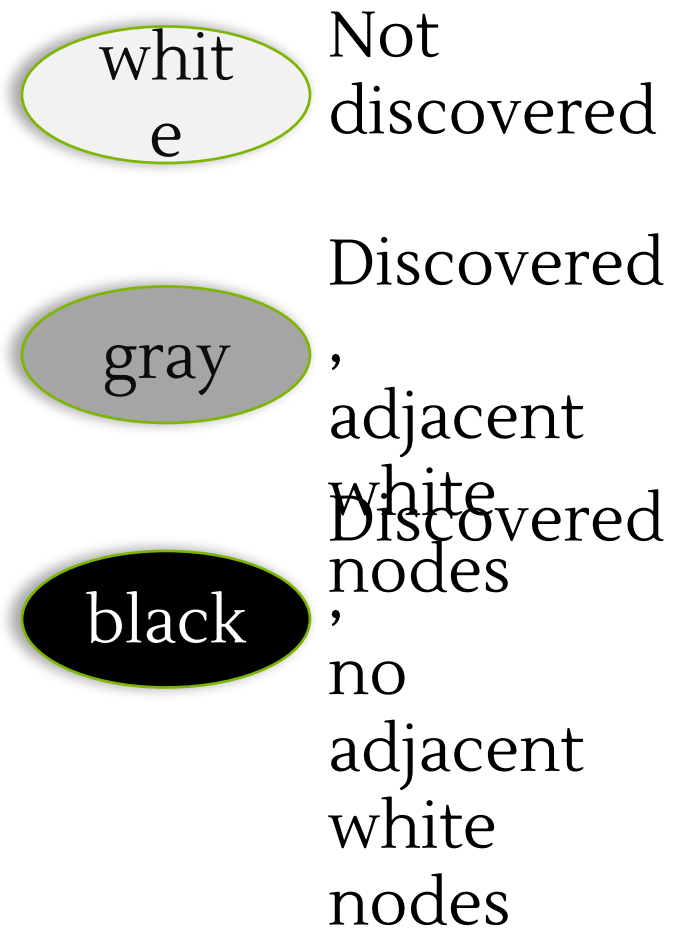
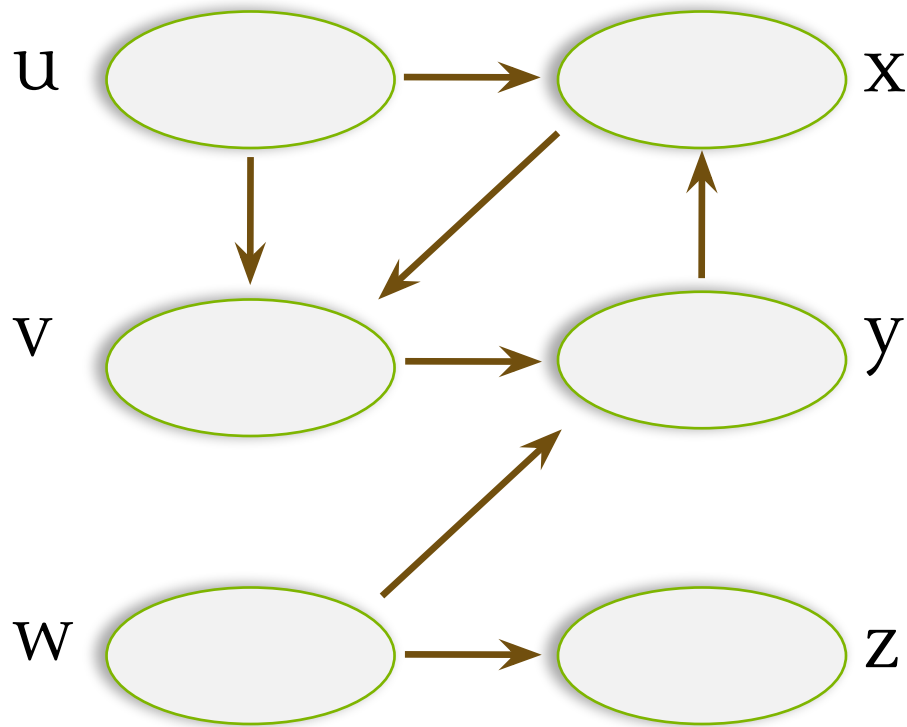
Depth-First Search (DFS)



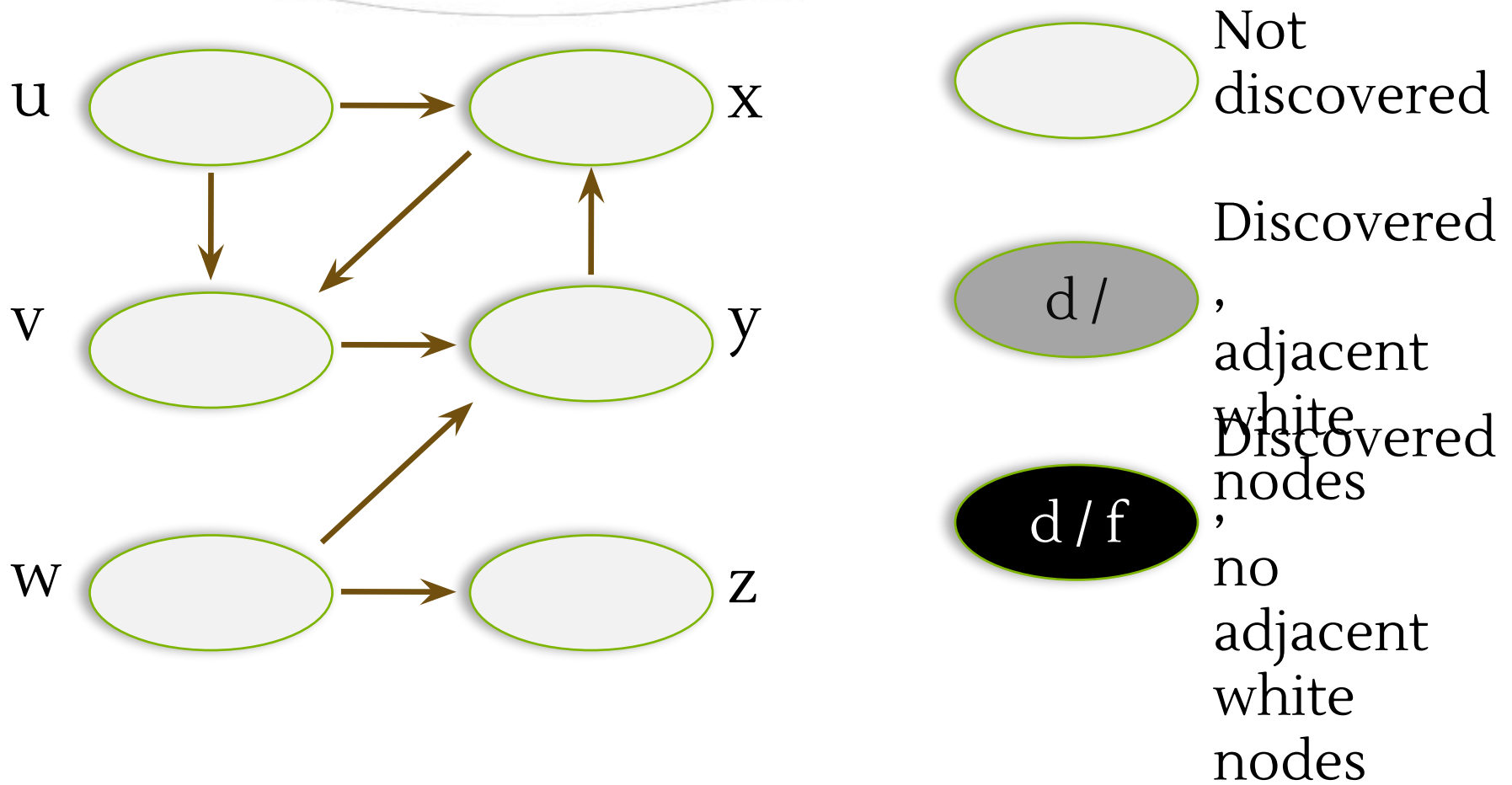
$d[u]$: when u is discovered
 $f[u]$: when searching adj of u is finished

1. $d[u] < f[u]$
2. $[d[u], f[u]]$ entirely contains $[d[v], f[v]]$
3. $[d[v], f[v]]$ and $[d[w], f[w]]$ are entirely disjoint

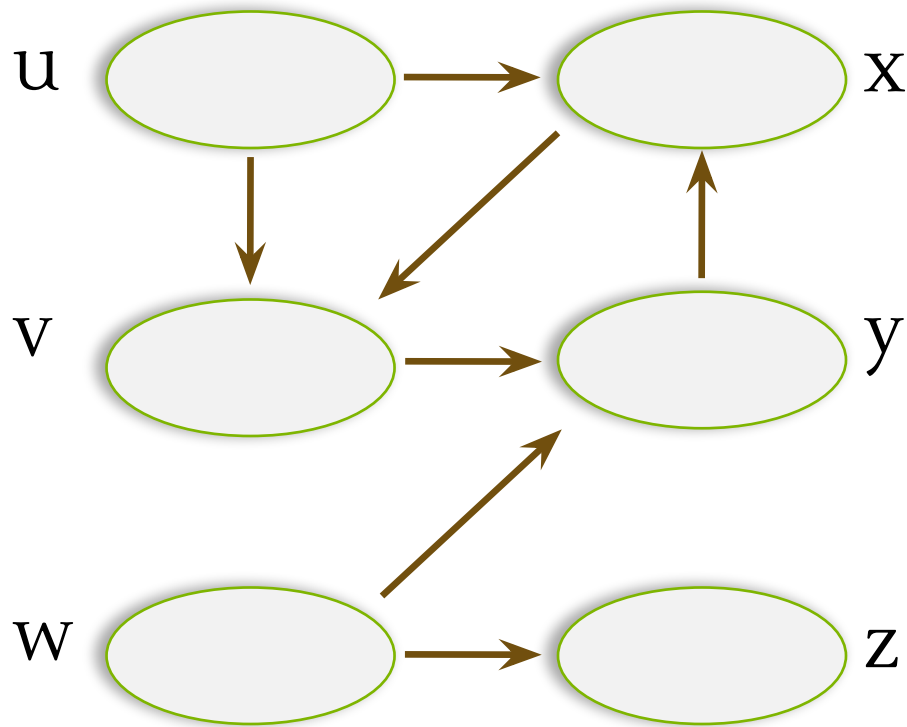
Depth-First Search (DFS)



Depth-First Search (DFS)



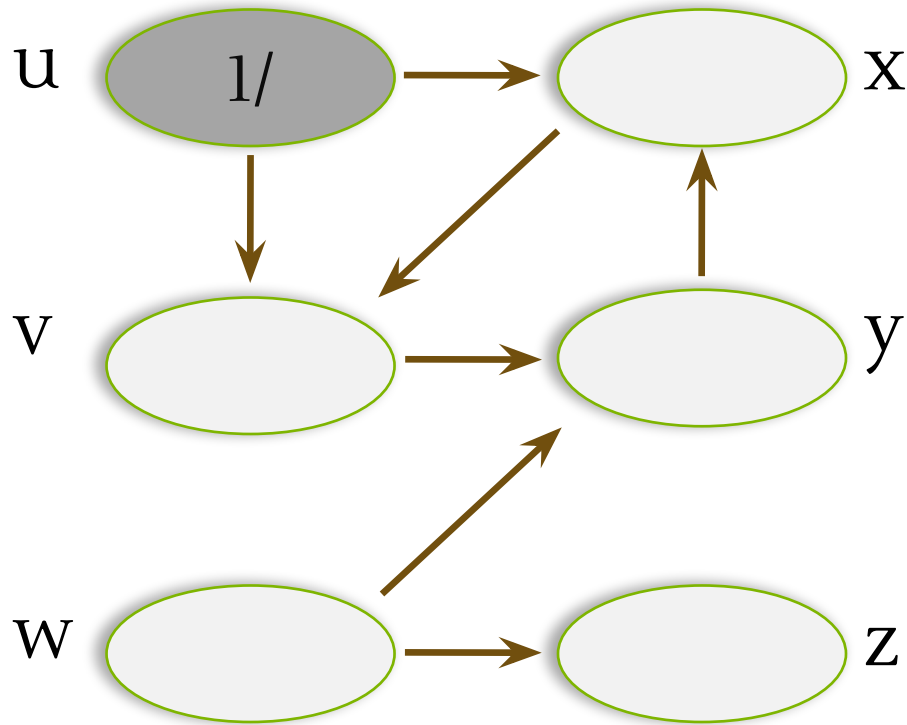
Depth-First Search (DFS)



DFS(G):

1. Initialization
for each $u \in V[G]$,
color[u] $\leftarrow$ white
 $\pi[u] \leftarrow \text{Nil}$
time $\leftarrow 0$

Depth-First Search (DFS)



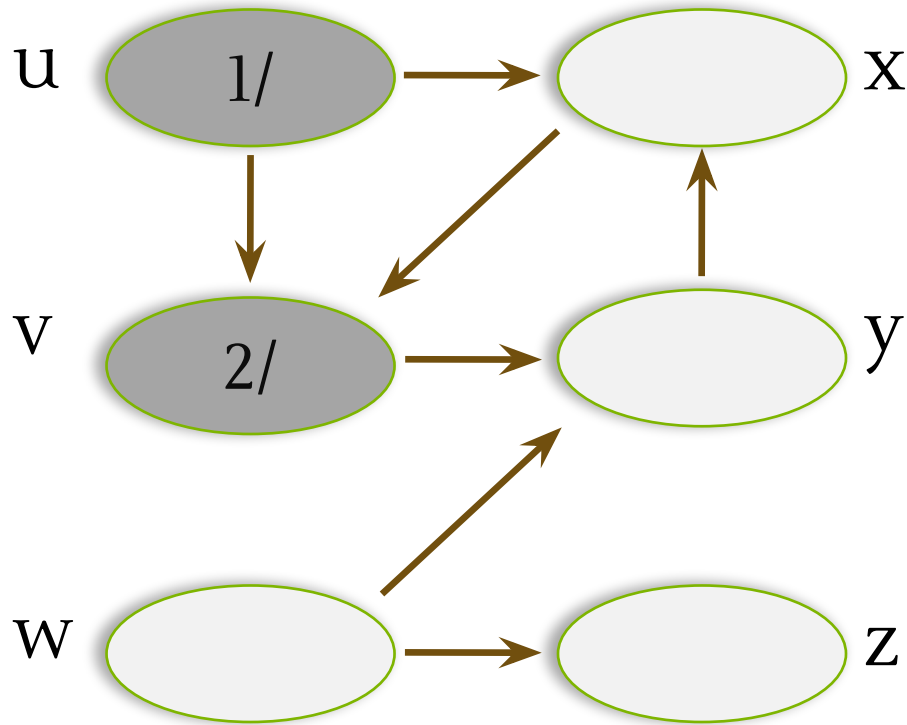
DFS(G):

1. Initialization
2. For each $u \in V[G]$
if $\text{color}[u] = \text{white}$
DFS-Visit(u)

DFS-Visit(u):

1. Initial Setting
 $\text{color}[u] \leftarrow \text{gray}$
 $d[u] \leftarrow \text{time} \leftarrow \text{time} + 1$

Depth-First Search (DFS)



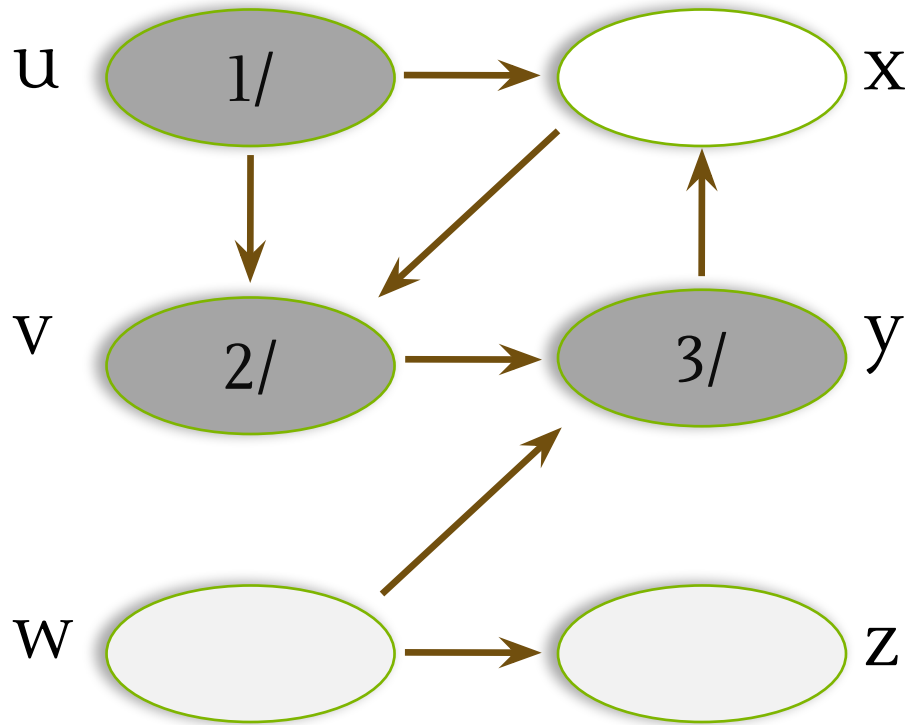
DFS(G):

1. Initialization
2. For each $u \in V[G]$
if $\text{color}[u] = \text{white}$
DFS-Visit(u)

DFS-Visit(u):

1. Initial Setting
2. for each adj v of white
 $\pi[v] \leftarrow u$
DFS-Visit[v]

Depth-First Search (DFS)



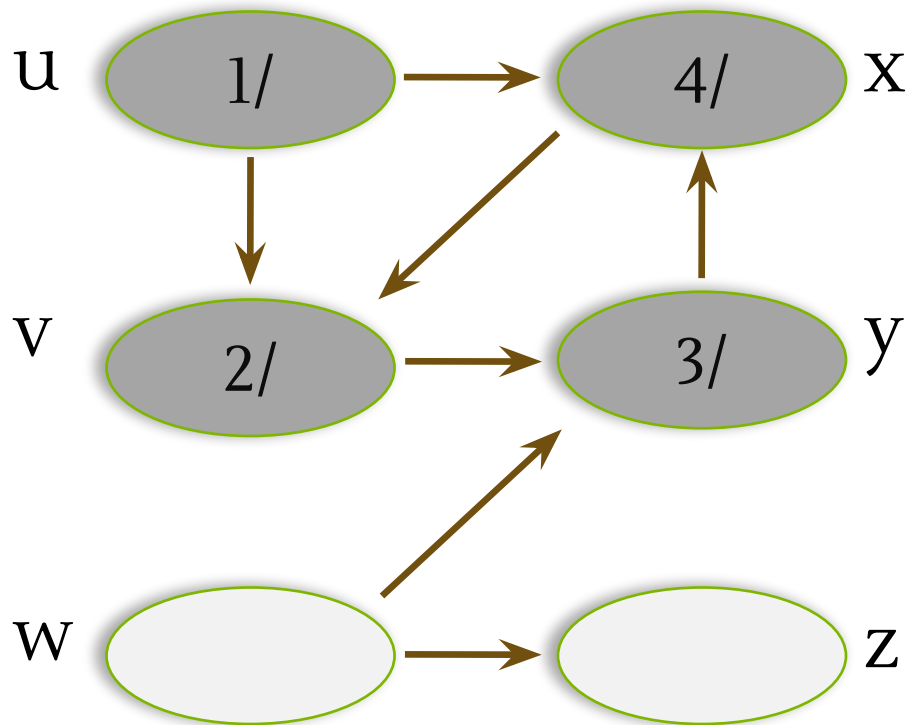
DFS(G):

1. Initialization
2. For each $u \in V[G]$
if $\text{color}[u] = \text{white}$
DFS-Visit(u)

DFS-Visit(u):

1. Initial Setting
2. for each adj v of u of white
 $\pi[v] \leftarrow u$
DFS-Visit(v)

Depth-First Search (DFS)



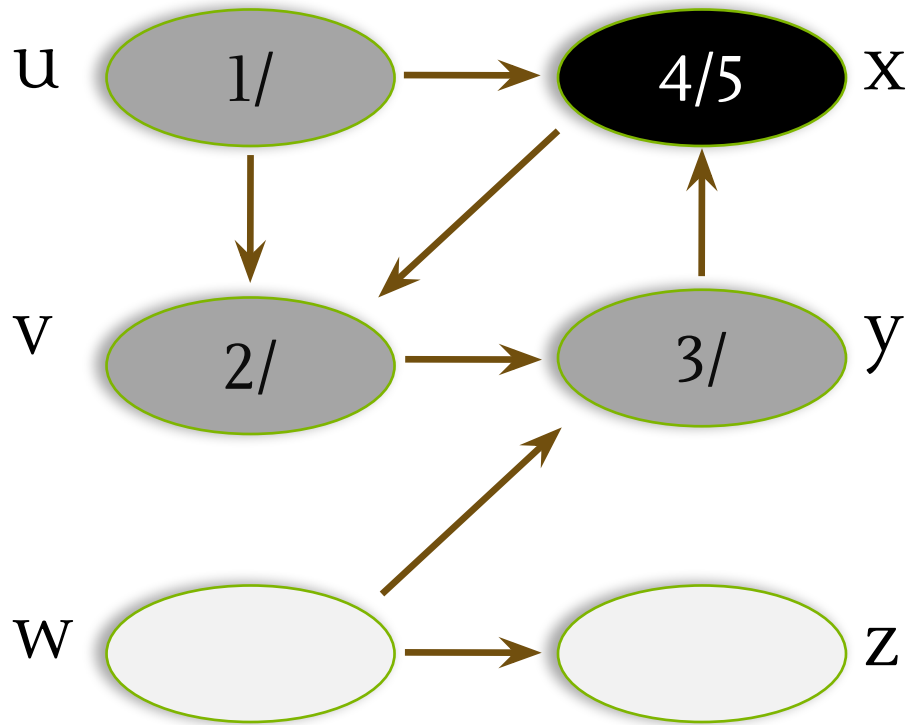
DFS(G):

1. Initialization
2. For each $u \in V[G]$
if $\text{color}[u] = \text{white}$
DFS-Visit(u)

DFS-Visit(u):

1. Initial Setting
2. for each adj v of u of white
 $\pi[v] \leftarrow u$
DFS-Visit(v)

Depth-First Search (DFS)



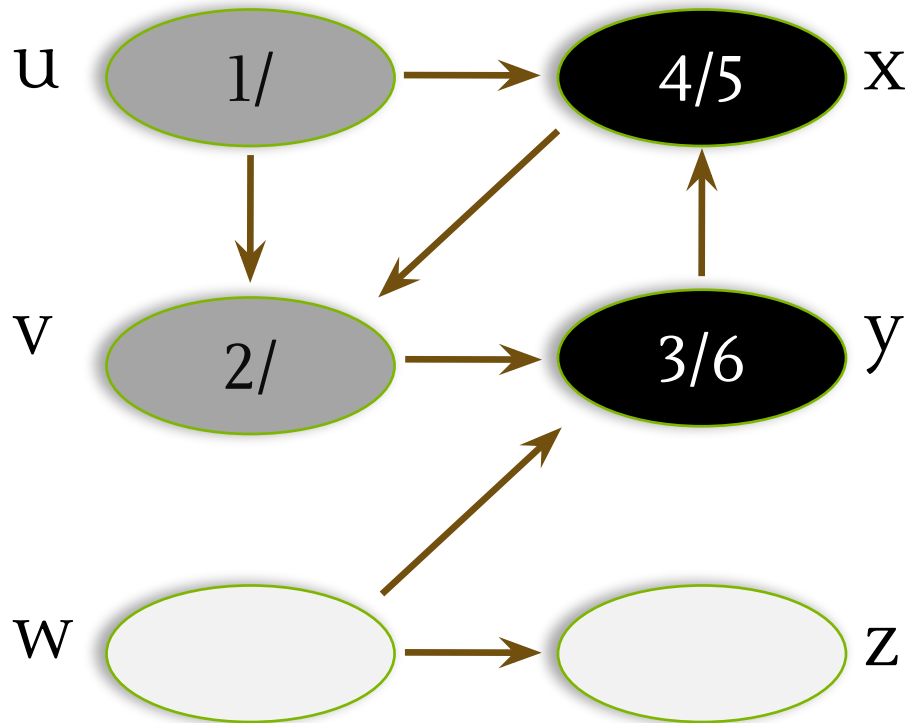
DFS(G):

1. Initialization
2. For each $u \in V[G]$
if $\text{color}[u] = \text{white}$
DFS-Visit(u)

DFS-Visit(u):

1. Initial Setting
2. Handling adj vertices
3. $\text{color}[u] \leftarrow \text{black}$
 $f[u] \leftarrow \text{time} \leftarrow \text{time} + 1$

Depth-First Search (DFS)



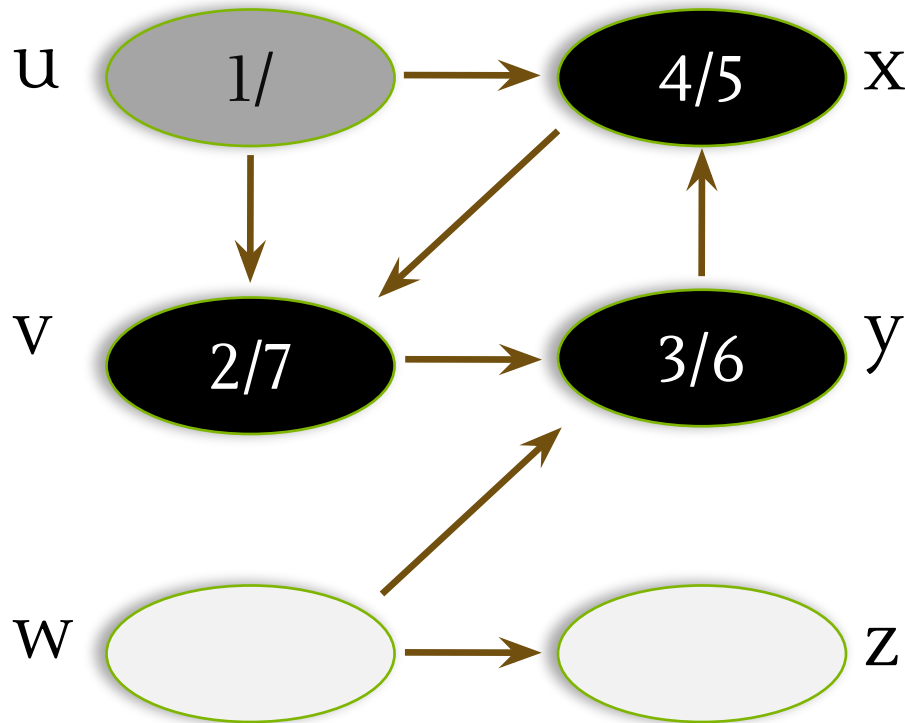
DFS(G):

1. Initialization
2. For each $u \in V[G]$
if $\text{color}[u] = \text{white}$
DFS-Visit(u)

DFS-Visit(u):

1. Initial Setting
2. Handling adj vertices
3. $\text{color}[u] \leftarrow \text{black}$
 $f[u] \leftarrow \text{time} \leftarrow \text{time} + 1$

Depth-First Search (DFS)



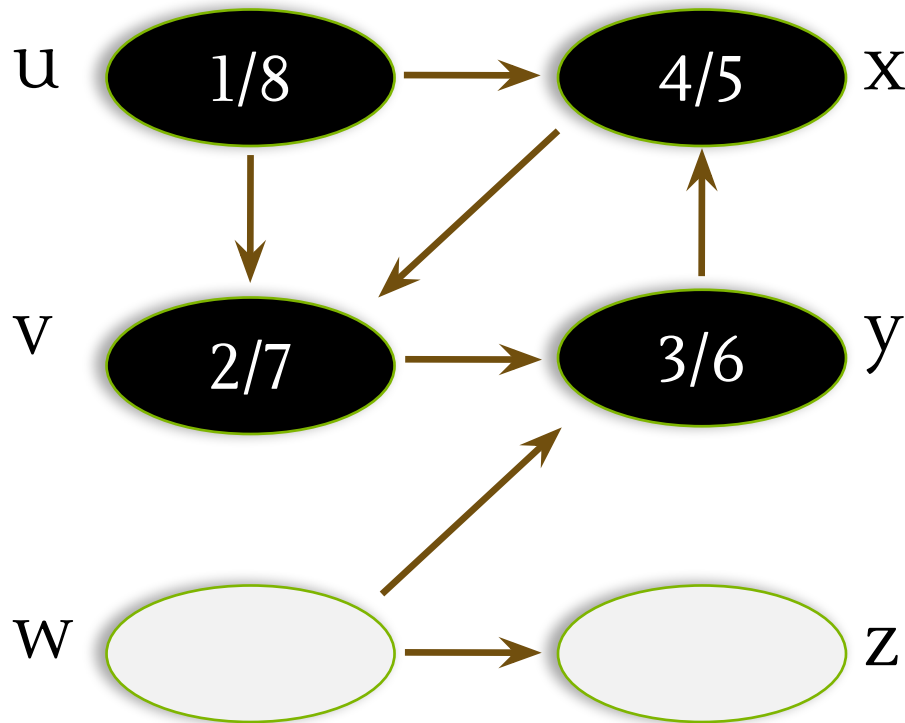
DFS(G):

1. Initialization
2. For each $u \in V[G]$
if $\text{color}[u] = \text{white}$
DFS-Visit(u)

DFS-Visit(u):

1. Initial Setting
2. Handling adj vertices
3. $\text{color}[u] \leftarrow \text{black}$
 $f[u] \leftarrow \text{time} \leftarrow \text{time} + 1$

Depth-First Search (DFS)



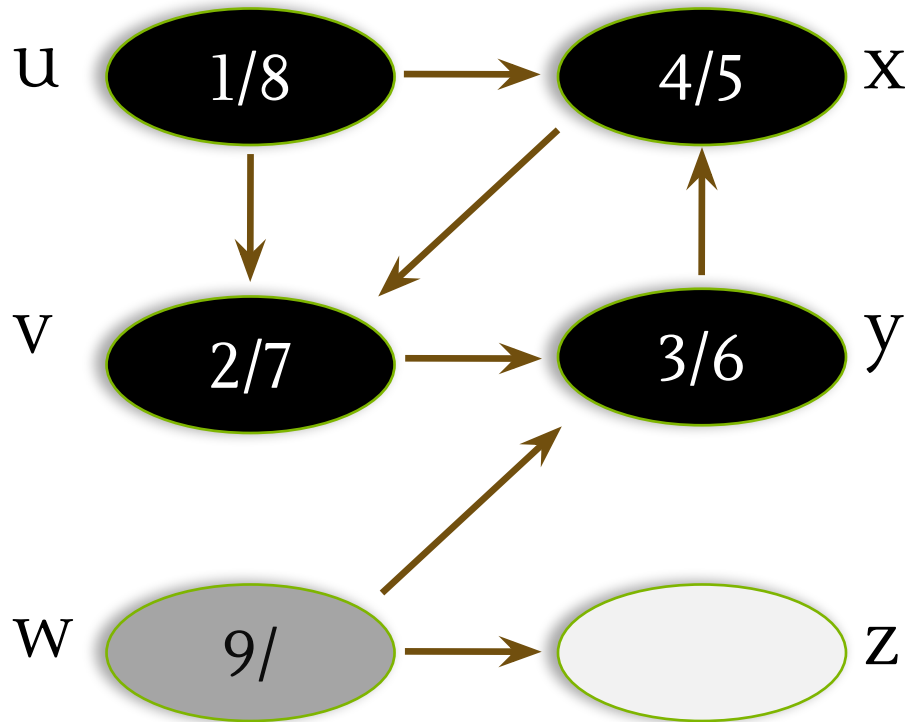
DFS(G):

1. Initialization
2. For each $u \in V[G]$
if color[u] = white
DFS-Visit(u)

DFS-Visit(u):

1. Initial Setting
2. Handling adj vertices
3. color[u] $\square$ black
f[u] $\square$ time $\square$ time + 1

Depth-First Search (DFS)



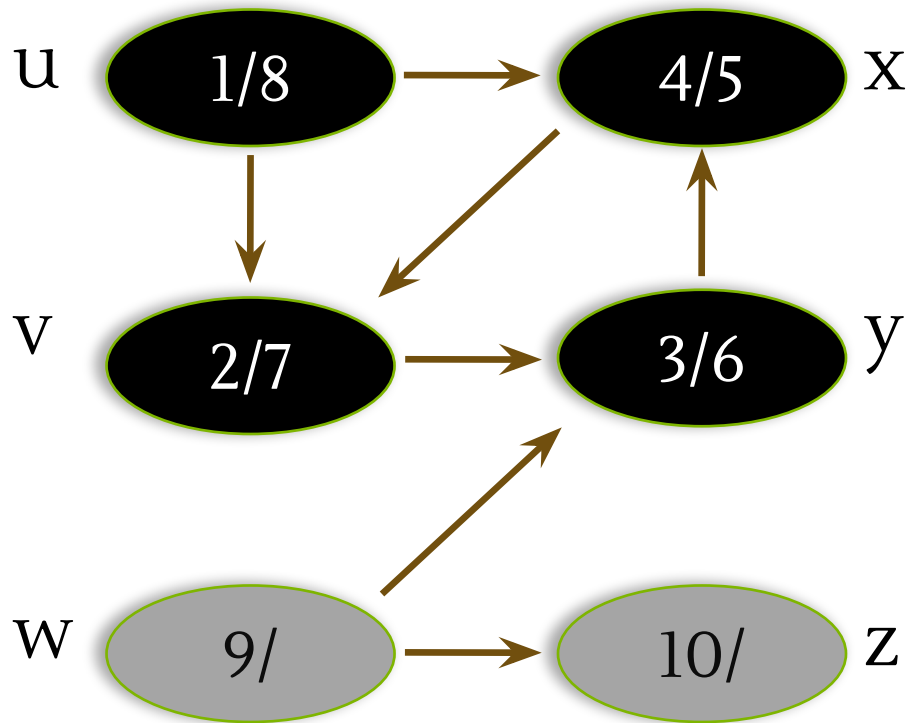
DFS(G):

1. Initialization
2. For each $u \in V[G]$
if $\text{color}[u] = \text{white}$
DFS-Visit(u)

DFS-Visit(u):

1. Initial Setting
2. Handling adj vertices
3. $\text{color}[u] \sqsubseteq \text{black}$
 $f[u] \sqsubseteq \text{time} \sqsubseteq \text{time} + 1$

Depth-First Search (DFS)



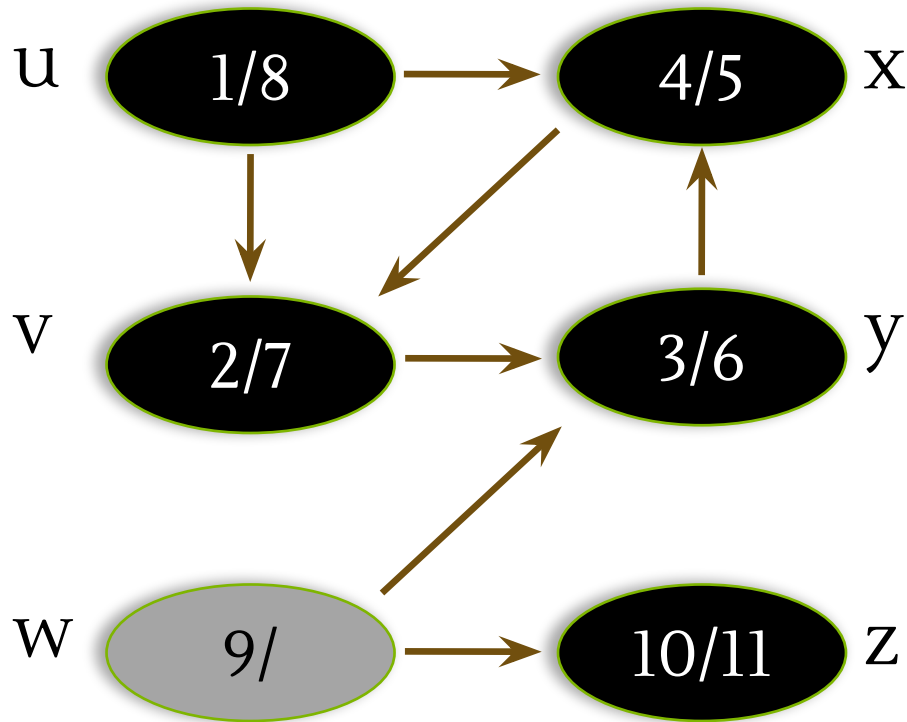
DFS(G):

1. Initialization
2. For each $u \in V[G]$
if $\text{color}[u] = \text{white}$
DFS-Visit(u)

DFS-Visit(u):

1. Initial Setting
2. Handling adj vertices
3. $\text{color}[u] \sqcap \text{black}$
 $f[u] \sqcap \text{time} \sqcap \text{time} + 1$

Depth-First Search (DFS)



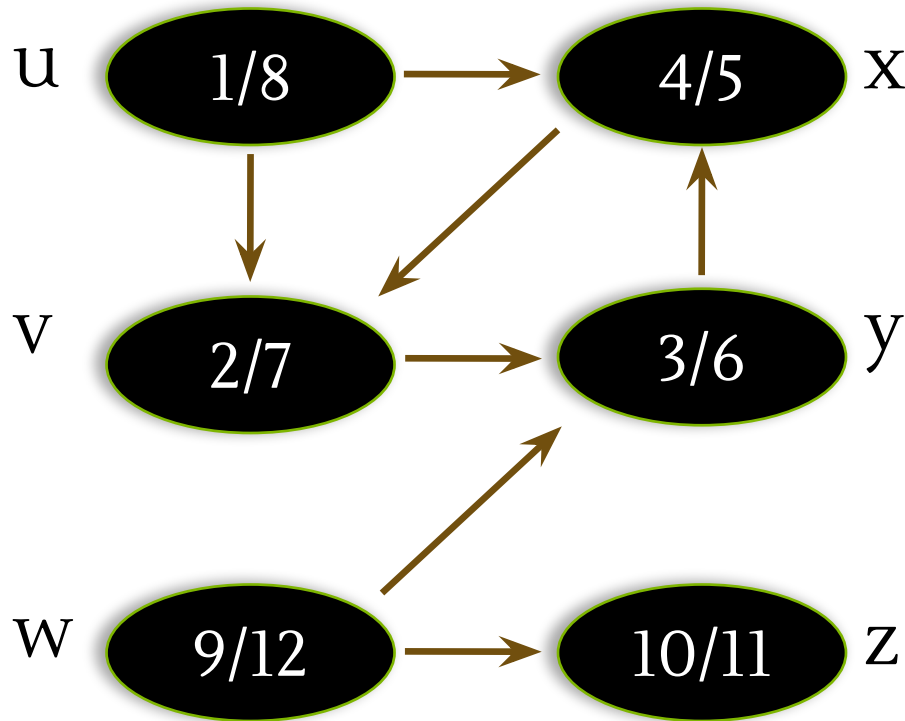
DFS(G):

1. Initialization
2. For each $u \in V[G]$
if $\text{color}[u] = \text{white}$
DFS-Visit(u)

DFS-Visit(u):

1. Initial Setting
2. Handling adj vertices
3. $\text{color}[u] \leftarrow \text{black}$
 $f[u] \leftarrow \text{time} \leftarrow \text{time} + 1$

Depth-First Search (DFS)



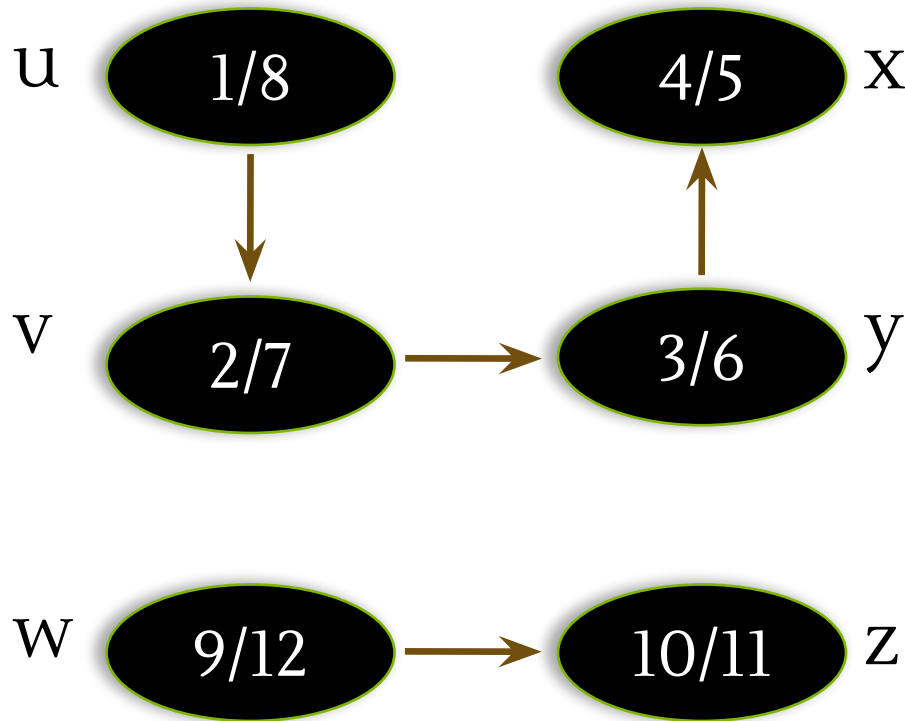
DFS(G):

1. Initialization
2. For each $u \in V[G]$
if $\text{color}[u] = \text{white}$
DFS-Visit(u)

DFS-Visit(u):

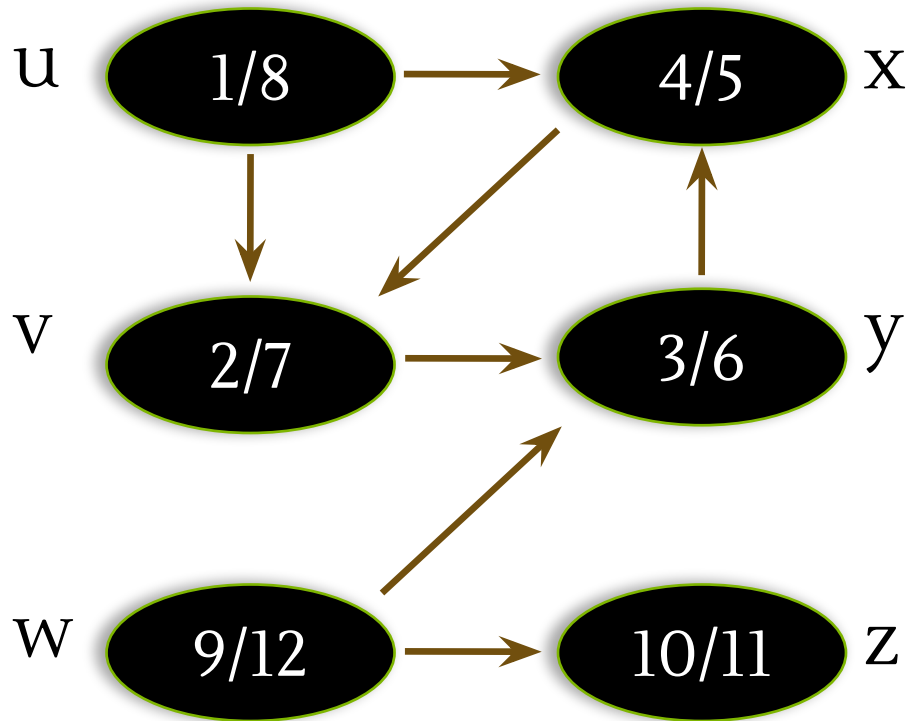
1. Initial Setting
2. Handling adj vertices
3. $\text{color}[u] \sqcap \text{black}$
 $f[u] \sqcap \text{time} \sqcap \text{time} + 1$

Depth-First Search (DFS)



DFS(G):
- construct a forest

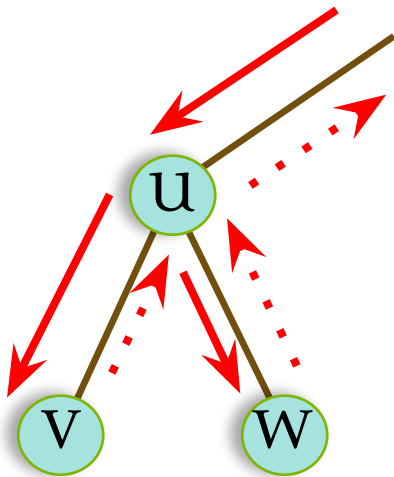
Depth-First Search (DFS)



DFS(G):

- Initialization: $O(|V|)$
 - Traversing vertices: $O(|V|)$
 - Scanning adjacent vertices: $O(|E|)$
- => total running time: $O(|V| + |E|)$

Depth-First Search (DFS)



1. $d[u] < f[u]$
2. $[d[u], f[u]]$ entirely contains $[d[v], f[v]]$
3. $[d[v], f[v]]$ and $[d[w], f[w]]$ are entirely disjoint

In a depth-first forest,
 v is a descendant of u
if and only if
 $d[u] < d[v] < f[v] < f[u]$

DFS Time and Space Complexity

- Time Complexity: $O(V + E)$
(V = nodes, E = edges)
- Space Complexity: $O(V)$ for the visited array and recursion stack.

Depth-First Search (DFS)

Applications

1. Cycle Detection in Graphs

- DFS is often used to check whether a graph has cycles — in directed or undirected graphs.

2. Topological Sorting

- Used in course prerequisite scheduling, build systems like make or CMake — whenever tasks have dependencies.

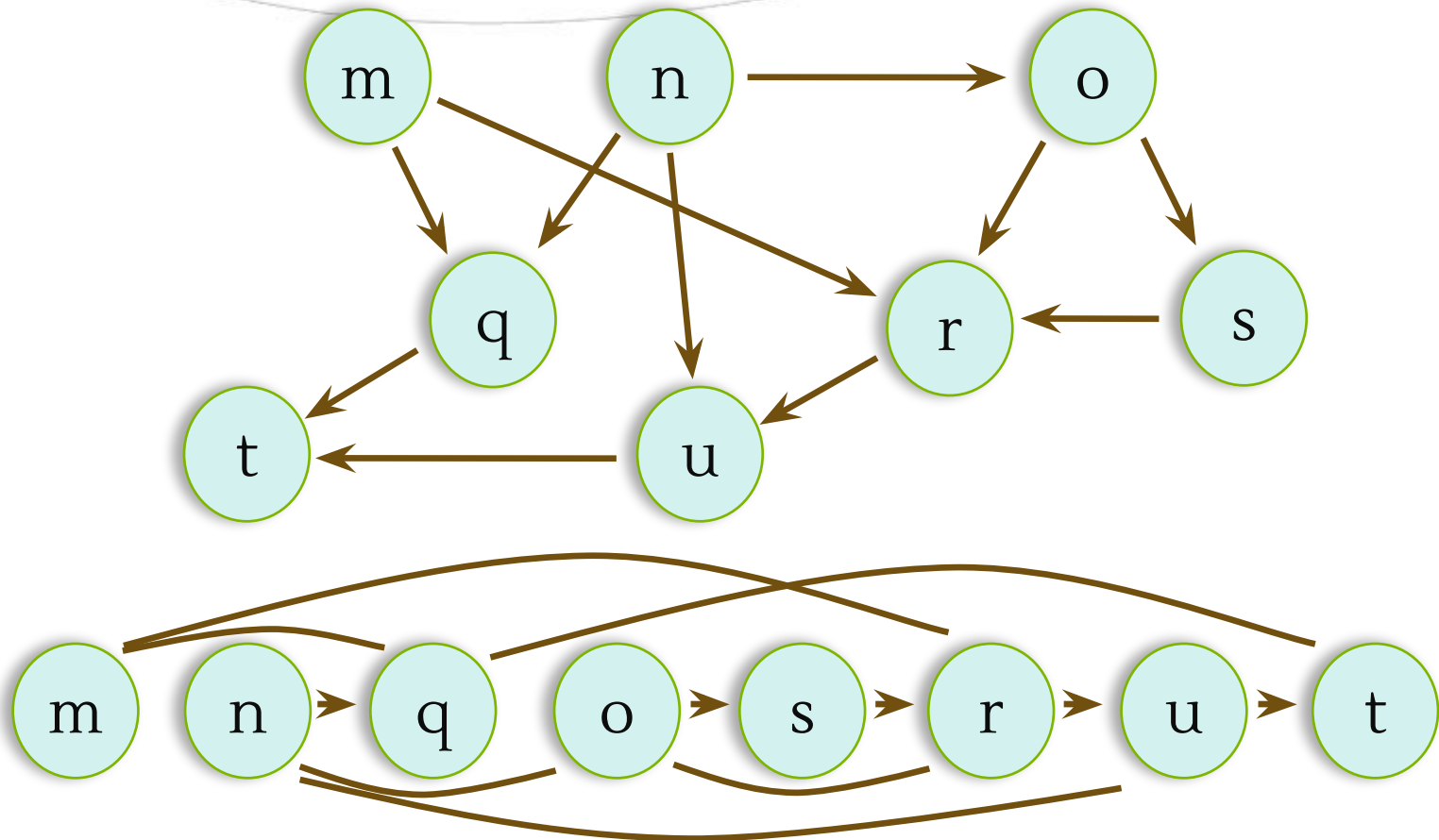
3. Connected Components in Graphs

- DFS can count the number of distinct components in an undirected graph.

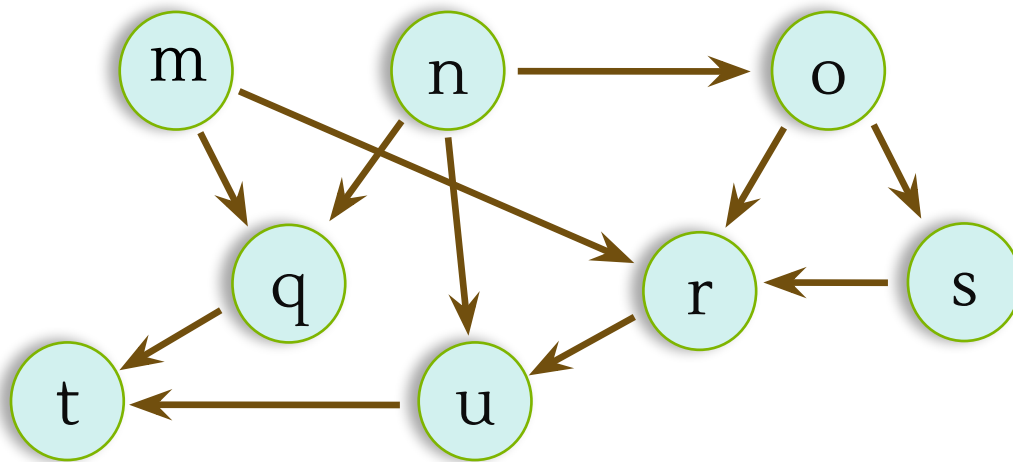
4. Solving Mazes & Puzzles

- DFS explores all paths deeply, which is ideal for maze-like problems, backtracking problems, and even games like Sudoku.

Topological Sorting



Topological Sorting



$$O(|V|^2 + |V||E|)$$

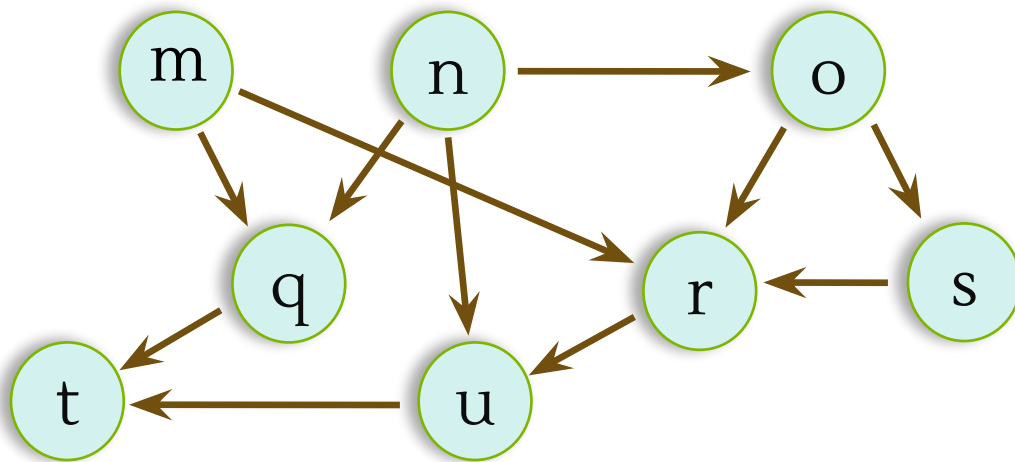
Or

$$O(|V|^2)$$

Brute-Force way

1. Find a vertex without edges.
2. Put it onto the front of the list, and remove it from G.
3. Remove all edges to the removed edge.
4. Repeat 1~3.

Topological Sorting



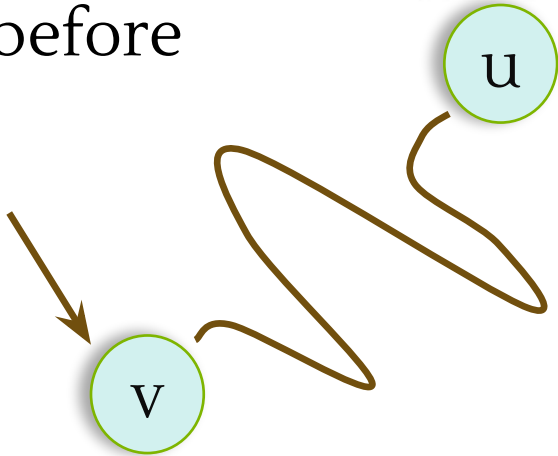
$$O(|V| + |E|)$$

Using DFS

1. Call DFS(G)
2. When a vertex is finished, put it onto the front of the list.

Topological Sorting

v enters the
list before
u?



Using DFS

1. Call $\text{DFS}(G)$
2. When a vertex is finished, put it onto the front of the list.

At $d[u]$:

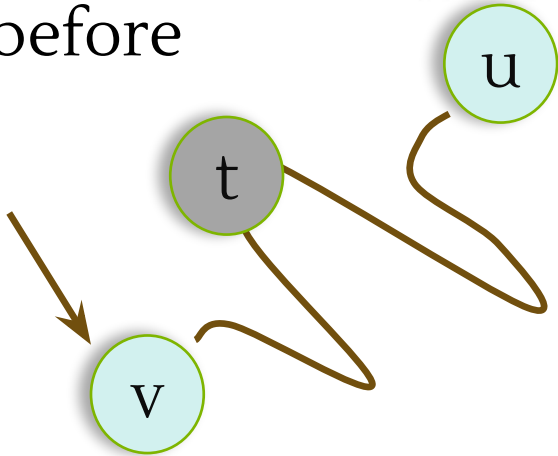
1) v is white: $d[u] < d[v] < f[u]$

2) v is black: $f[v] < d[u]$

3) v is gray: $d[v] < d[u] < f[v]$

Topological Sorting

v enters the
list before
u?



Using DFS

1. Call DFS(G)
2. When a vertex is finished, put it onto the front of the list.

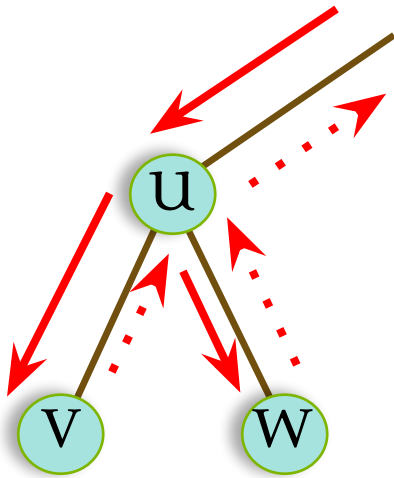
1) v is white: $d[u] < d[v]$

At $d[u]$:

t is gray: $d[t] < d[u] < f[t]$

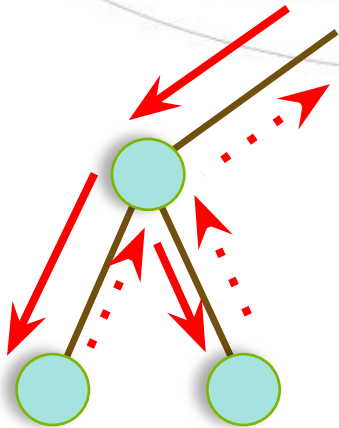
t is black: $f[t] < d[u]$

Topological Sorting



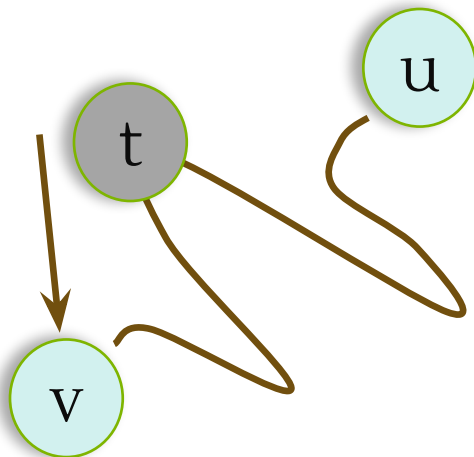
1. If v is a descendant,
 $d[u] < d[v] < f[v] < f[u]$
2. If v is an ancestor,
 $d[v] < d[u] < f[u] < f[v]$
3. Otherwise,
 $d[v] < f[v] < d[u] < f[u]$
or
 $d[u] < f[u] < d[v] < f[v]$

Topological Sorting



1. If v is a descendant,
 $d[u] < d[v] < f[v] < f[u]$
2. If v is an ancestor,
 $d[v] < d[u] < f[u] < f[v]$
3. Otherwise,
 $d[v] < f[v] < d[u] < f[u]$ or $d[u] < f[u] < d[v] < f[v]$

Contradiction:
G is not
acyclic.



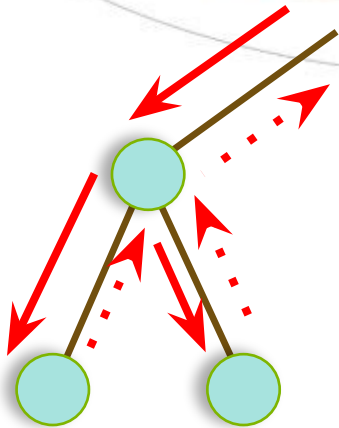
At $d[u]$:

1) v is white: $d[u] < d[v]$

t is gray: $d[t] < d[u] < f[t]$

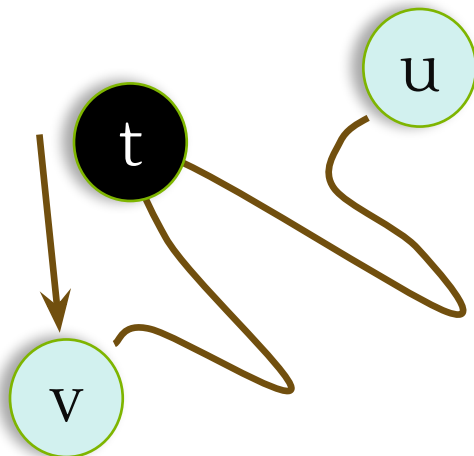
t is black: $f[t] < d[u]$

Topological Sorting



1. If v is a descendant,
 $d[u] < d[v] < f[v] < f[u]$
2. If v is an ancestor,
 $d[v] < d[u] < f[u] < f[v]$
3. Otherwise,
 $d[v] < f[v] < d[u] < f[u]$ or $d[u] < f[u] < d[v] < f[v]$

Contradiction:
 v should be
black.



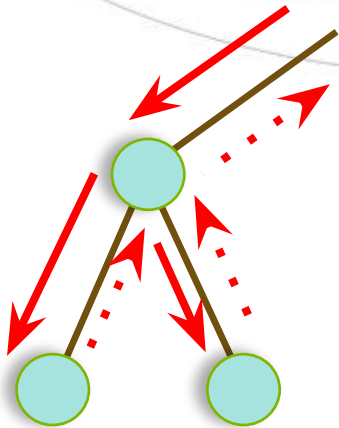
At $d[u]$:

1) v is white: $d[u] < d[v]$

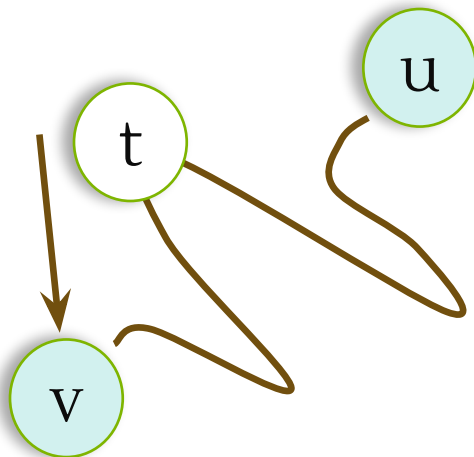
t is gray: $d[t] < d[u] < f[t]$

t is black: $f[t] < d[u]$

Topological Sorting



1. If v is a descendant,
 $d[u] < d[v] < f[v] < f[u]$
2. If v is an ancestor,
 $d[v] < d[u] < f[u] < f[v]$
3. Otherwise,
 $d[v] < f[v] < d[u] < f[u]$ or $d[u] < f[u] < d[v] < f[v]$

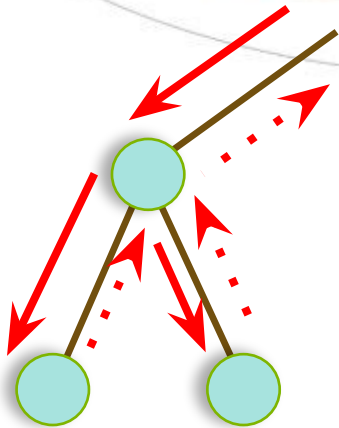


At $d[u]$:

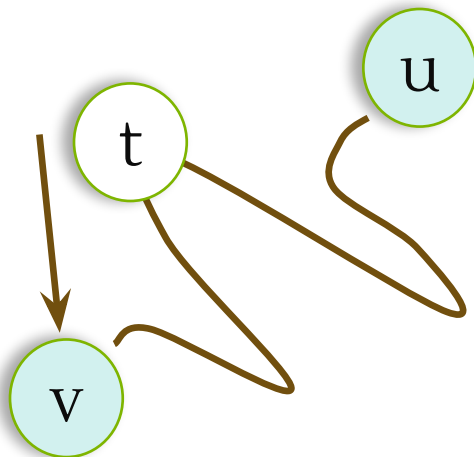
1) v is white: $d[u] < d[v]$

t is white.

Topological Sorting



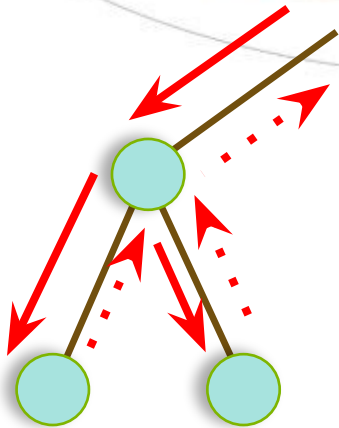
1. If v is a descendant,
 $d[u] < d[v] < f[v] < f[u]$
2. If v is an ancestor,
 $d[v] < d[u] < f[u] < f[v]$
3. Otherwise,
 $d[v] < f[v] < d[u] < f[u]$ or $d[u] < f[u] < d[v] < f[v]$



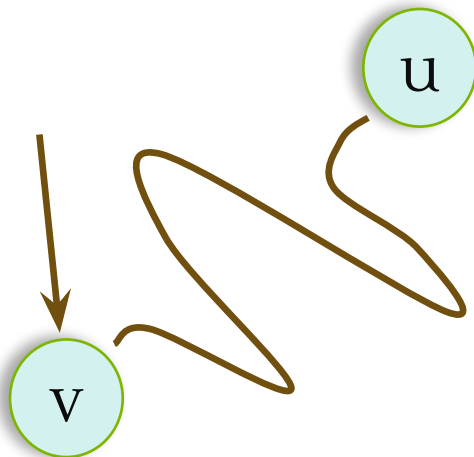
At $d[u]$:

1) v is white: $d[u] < d[v] < f[u]$
 t is white.

Topological Sorting



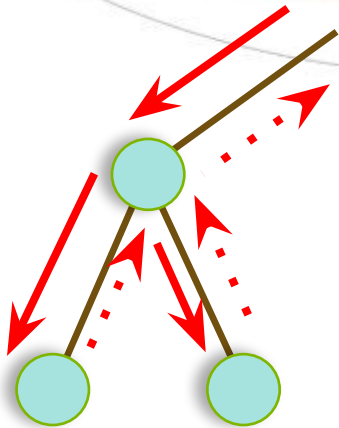
1. If v is a descendant,
 $d[u] < d[v] < f[v] < f[u]$
 2. If v is an ancestor,
 $d[v] < d[u] < f[u] < f[v]$
 3. Otherwise,
 $d[v] < f[v] < d[u] < f[u]$ or $d[u] < f[u] < d[v] < f[v]$
- v will enter the list before u.*



At $d[u]$:

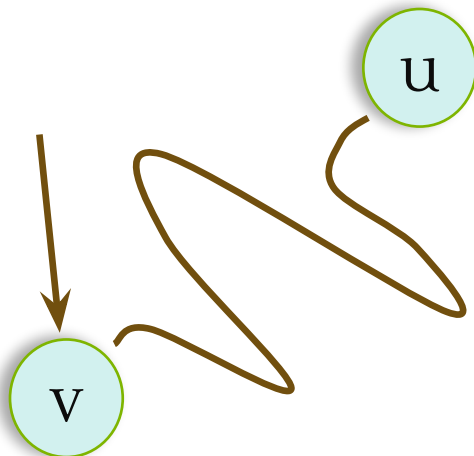
- 1) v is white: $d[u] < d[v] < f[u]$
- 2) v is black: $f[v] < d[u]$
- 3) v is gray: $d[v] < d[u] < f[v]$

Topological Sorting



1. If v is a descendant,
 $d[u] < d[v] < f[v] < f[u]$
2. If v is an ancestor,
 $d[v] < d[u] < f[u] < f[v]$
3. Otherwise,
 $d[v] < f[v] < d[u] < f[u]$ or $d[u] < f[u] < d[v] < f[v]$

v is already in the list.



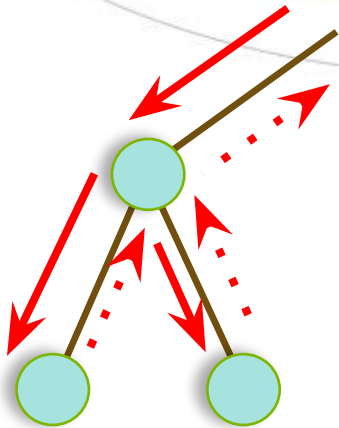
At $d[u]$:

1) v is white: $d[u] < d[v] < f[u]$

2) v is black: $f[v] < d[u]$

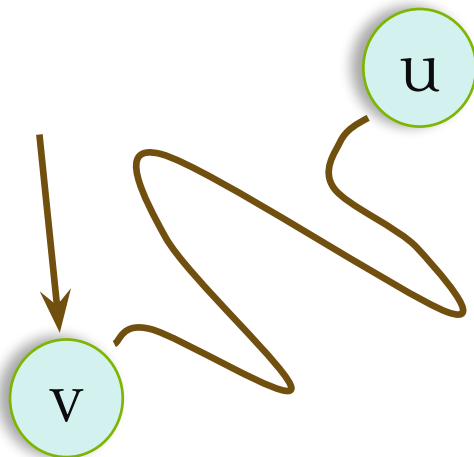
3) v is gray: $d[v] < d[u] < f[v]$

Topological Sorting



1. If v is a descendant,
 $d[u] < d[v] < f[v] < f[u]$
2. If v is an ancestor,
 $d[v] < d[u] < f[u] < f[v]$
3. Otherwise,
 $d[v] < f[v] < d[u] < f[u]$ or $d[u] < f[u] < d[v] < f[v]$

Contradiction:
 G is not
acyclic.



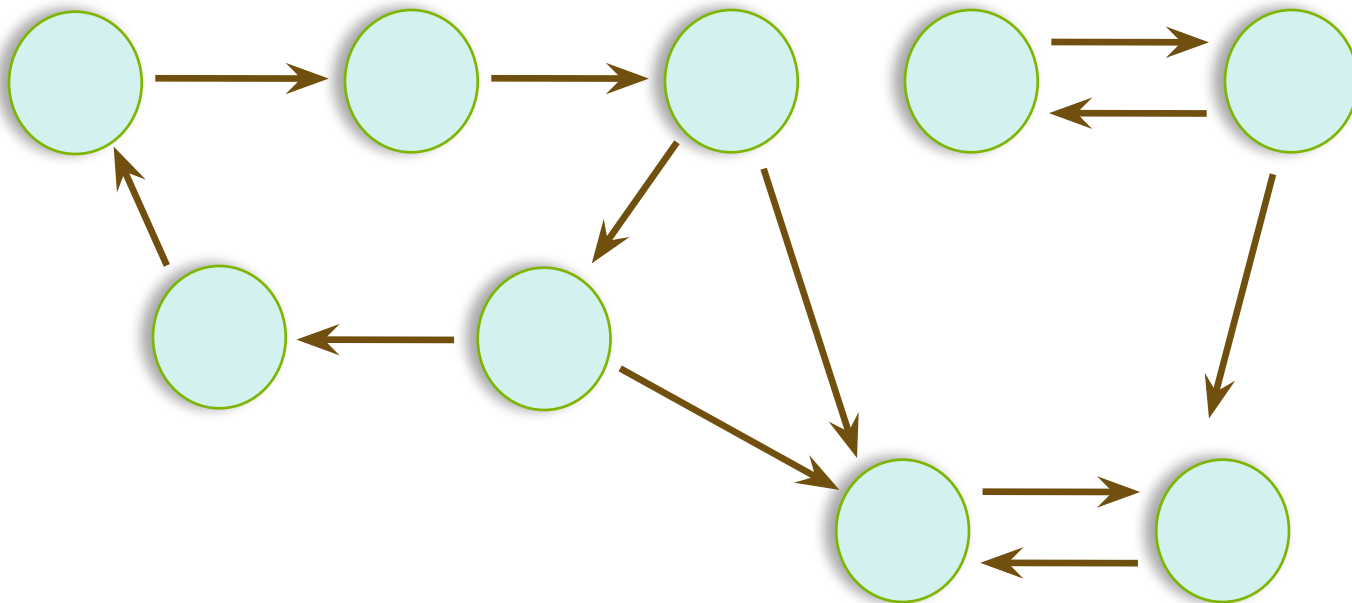
At $d[u]$:

1) v is white: $d[u] < d[v] < f[u]$

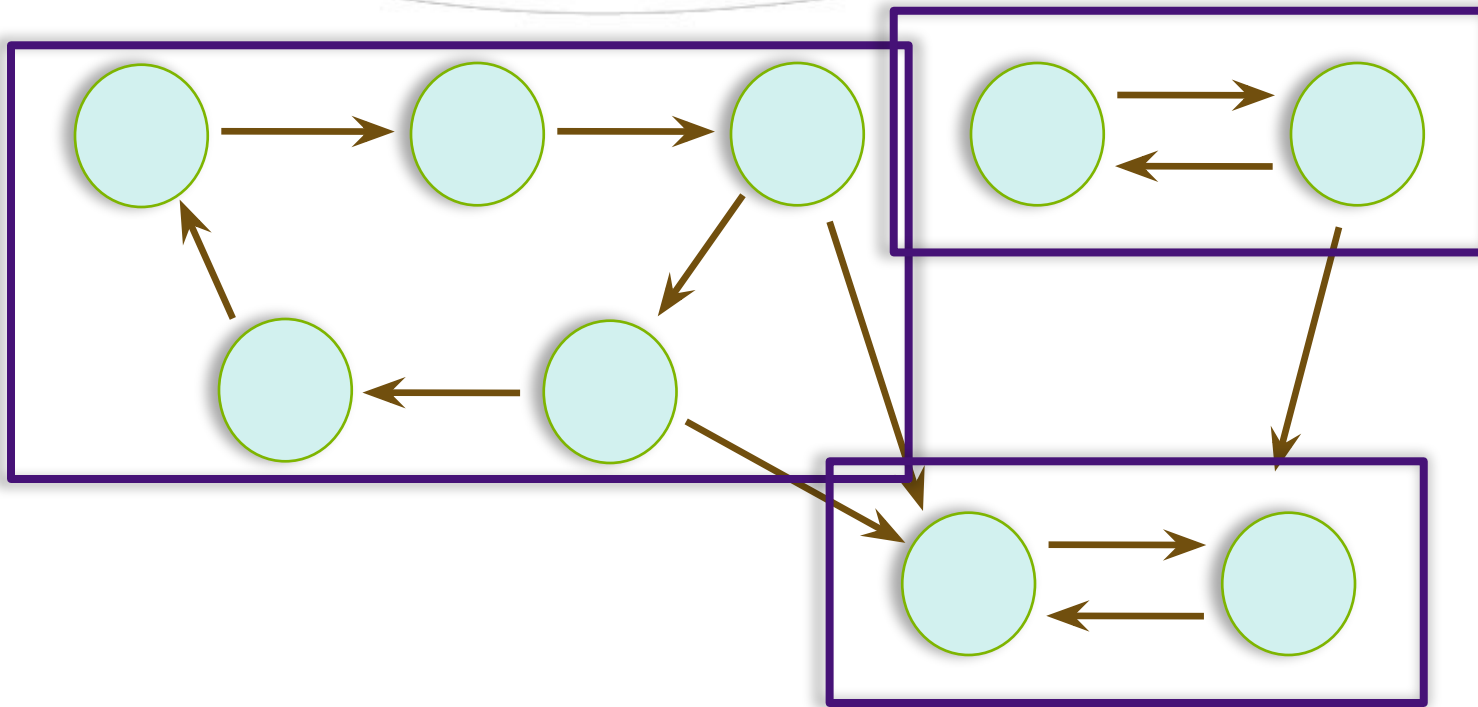
2) v is black: $f[v] < d[u]$

3) v is gray: $d[v] < d[u] < f[v]$

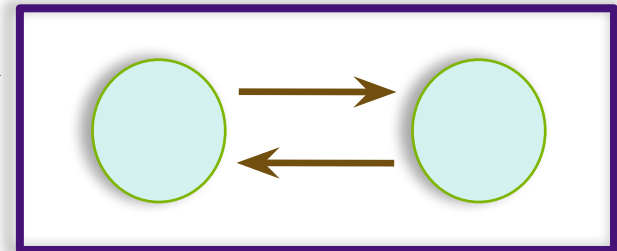
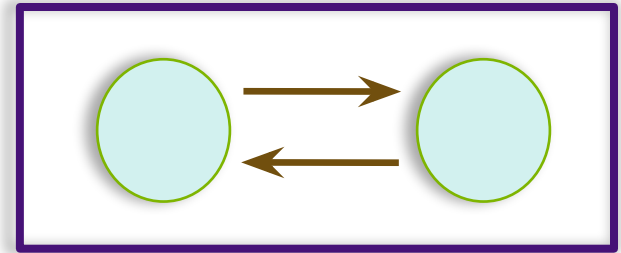
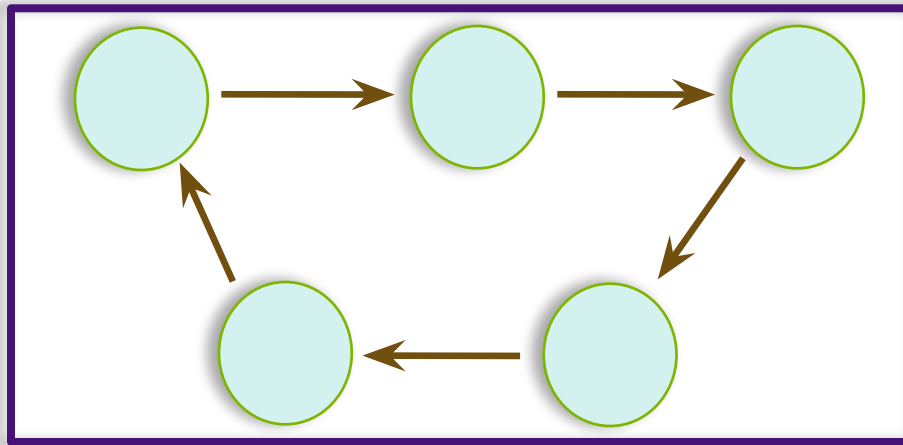
Strongly Connected Component



Strongly Connected Component

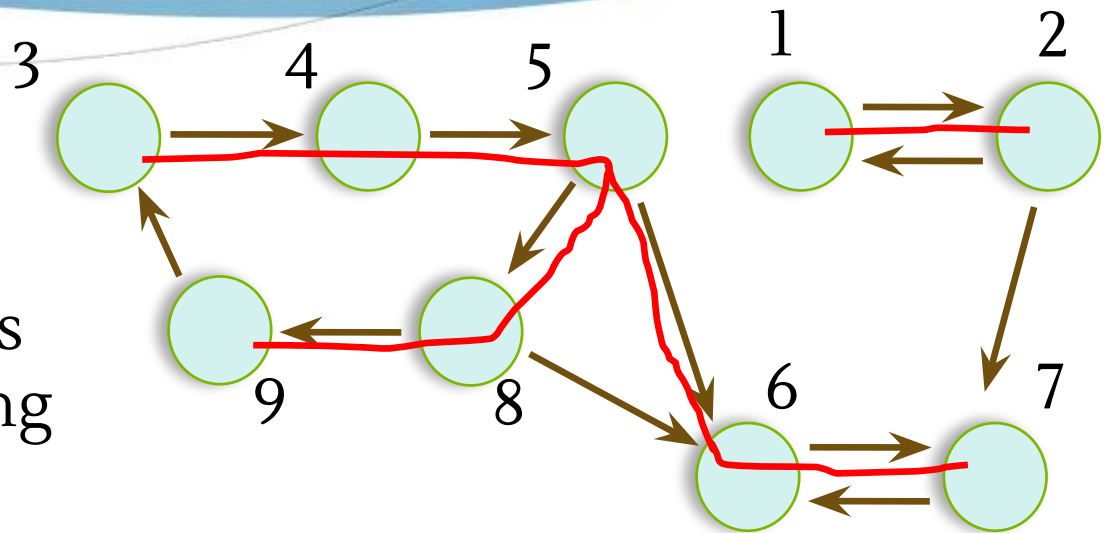


Strongly Connected Component



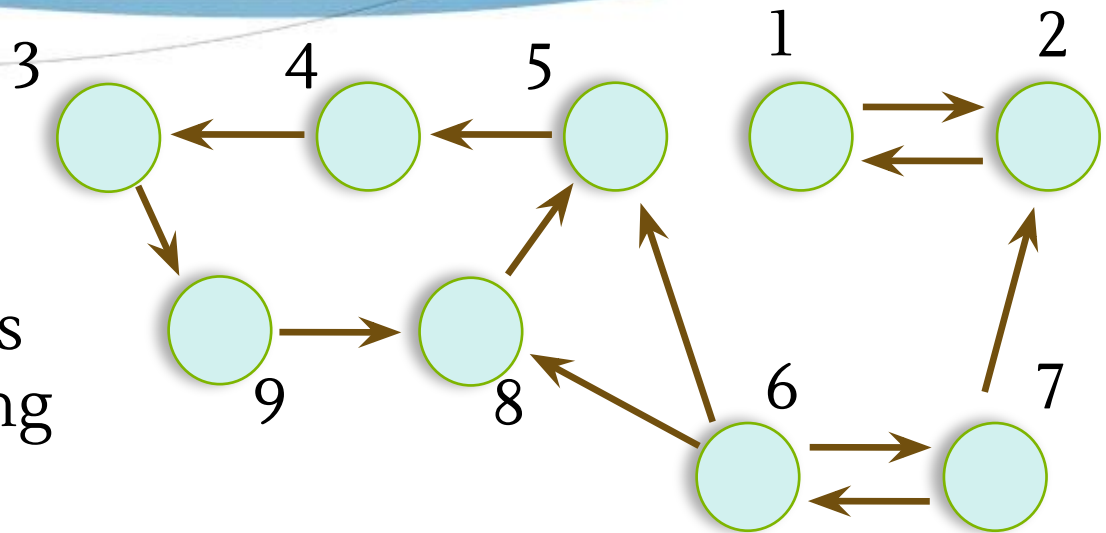
Strongly Connected Component

1. Call DFS(G)
2. Arrange the vertices in order of decreasing $f(u)$



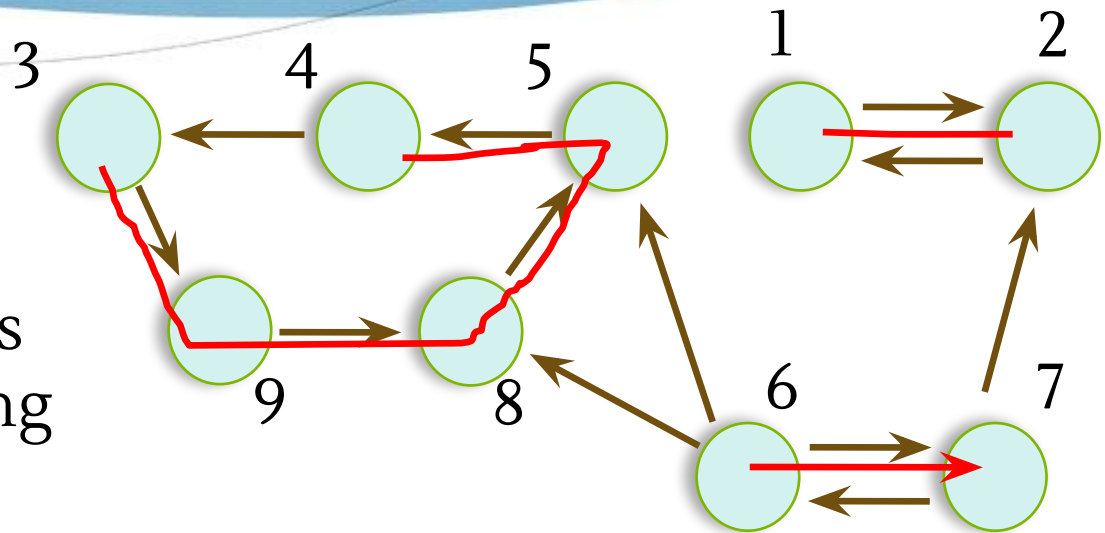
Strongly Connected Component

1. Call DFS(G)
2. Arrange the vertices in order of decreasing $f(u)$
3. Compute G^T



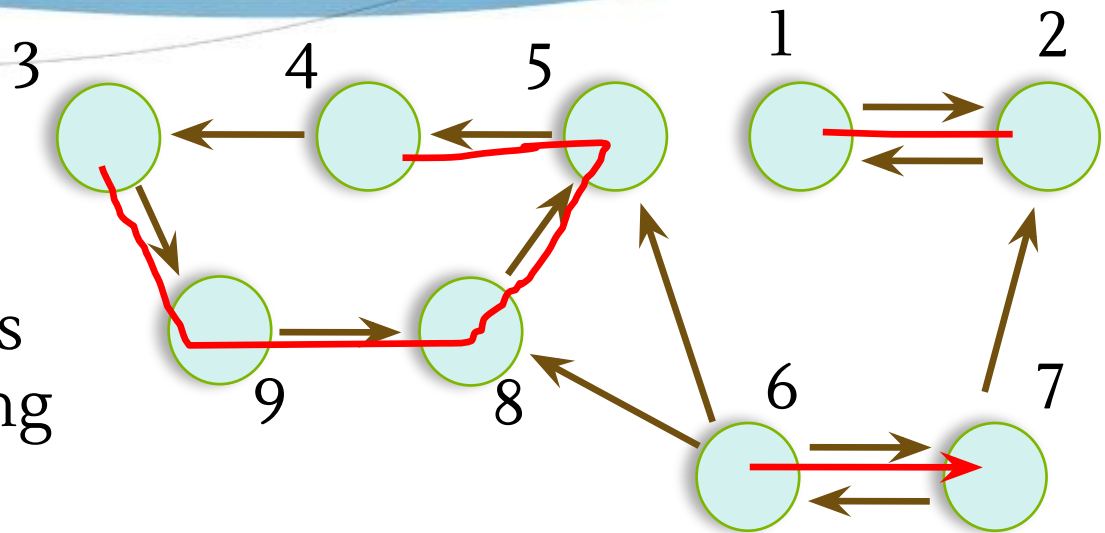
Strongly Connected Component

1. Call DFS(G)
2. Arrange the vertices in order of decreasing $f(u)$
3. Compute G^T
4. Run DFS(G^T)

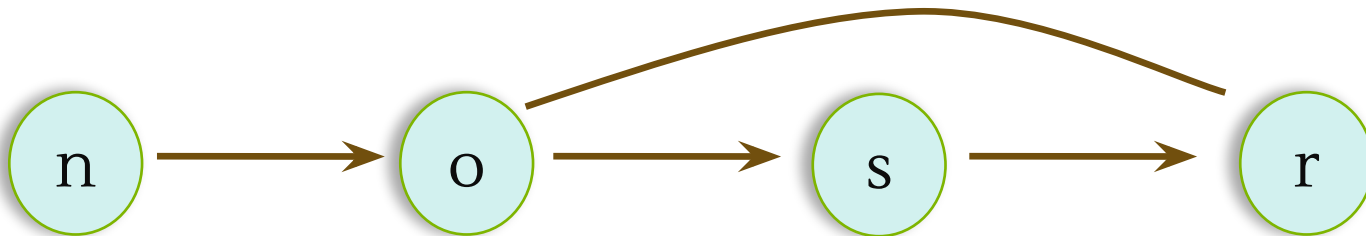
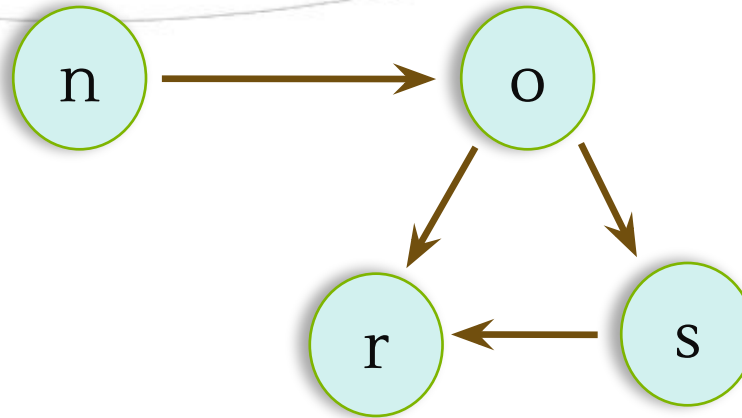


Strongly Connected Component

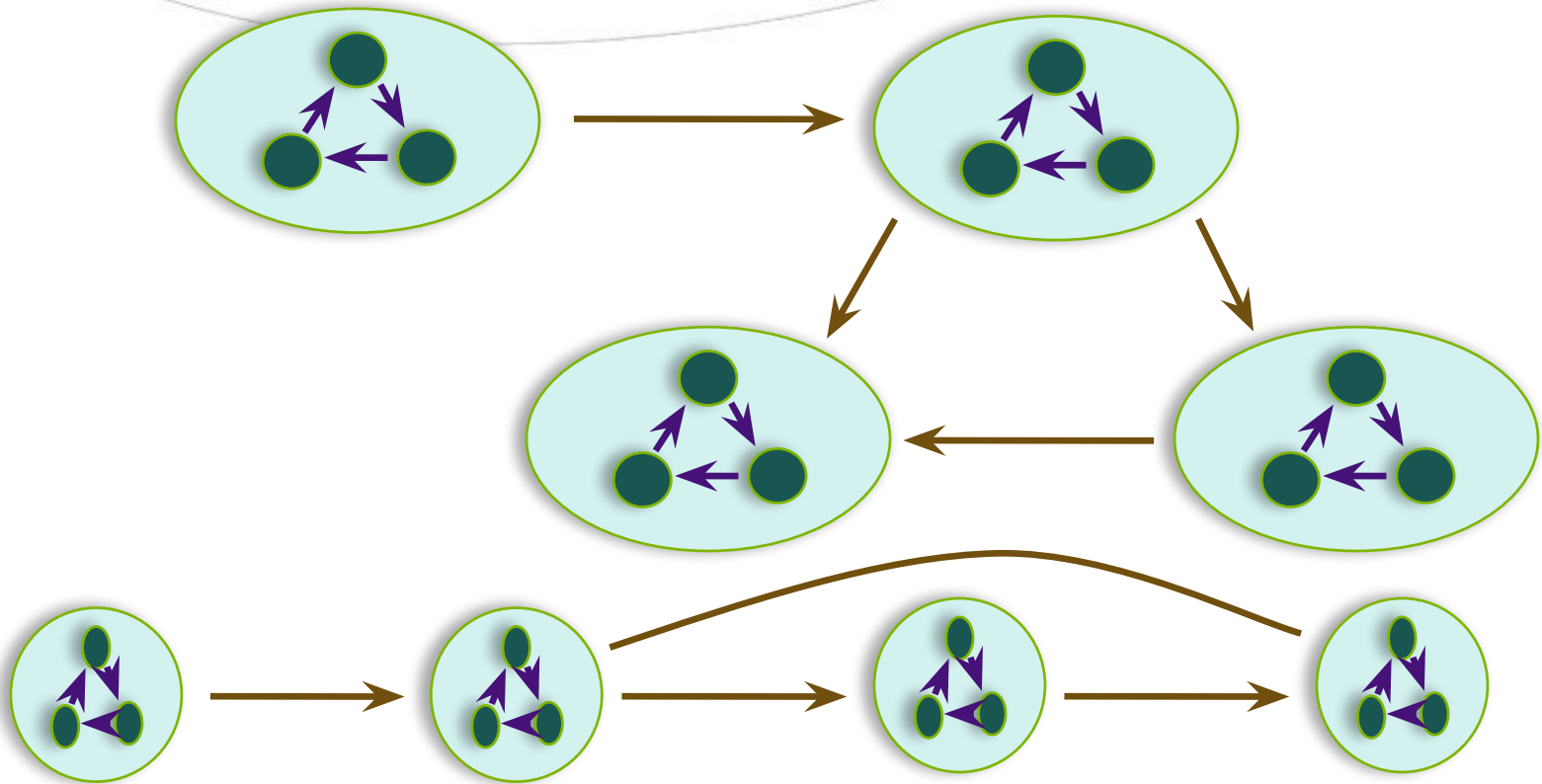
1. Call DFS(G)
2. Arrange the vertices in order of decreasing $f(u)$
3. Compute G^T
4. Run DFS(G^T)



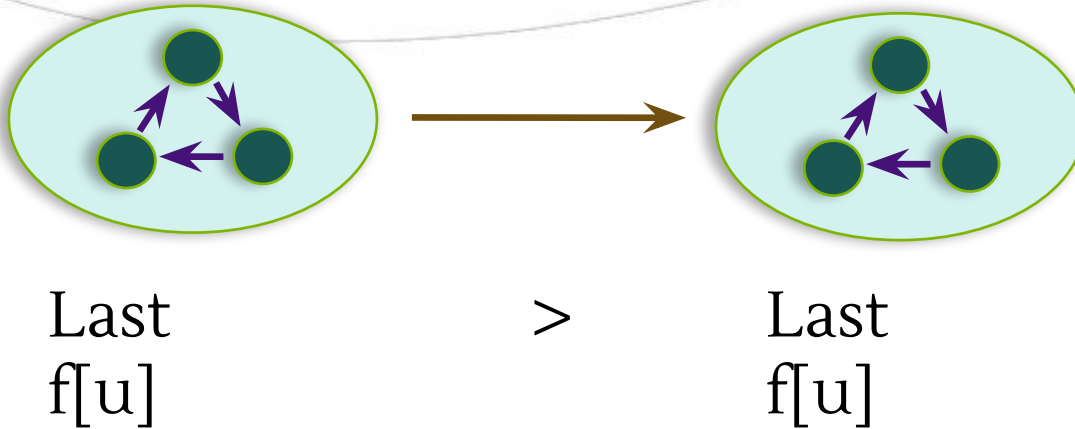
Topological Sorting



Strongly Connected Component

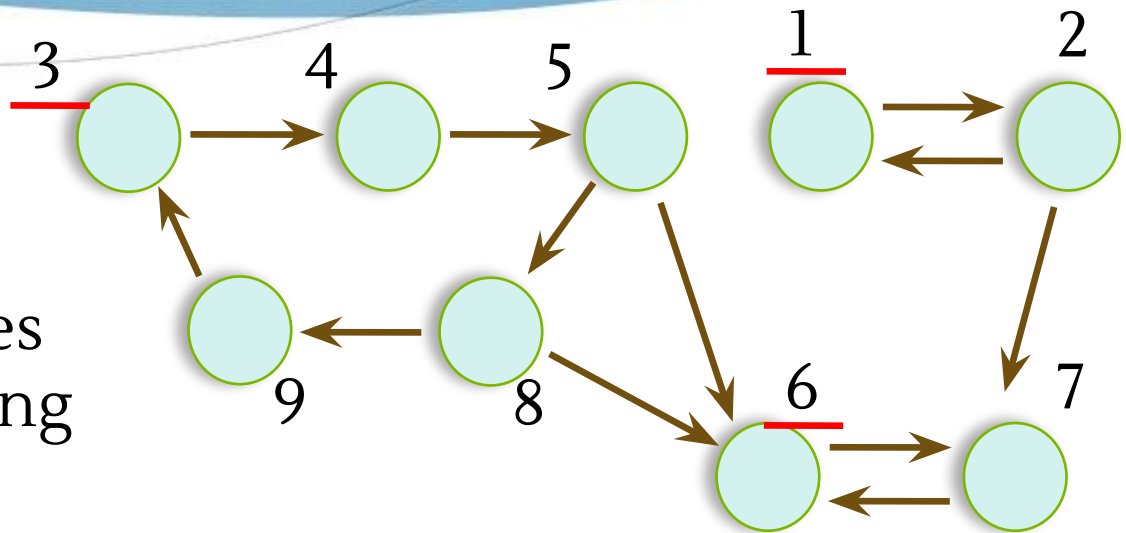


Strongly Connected Component

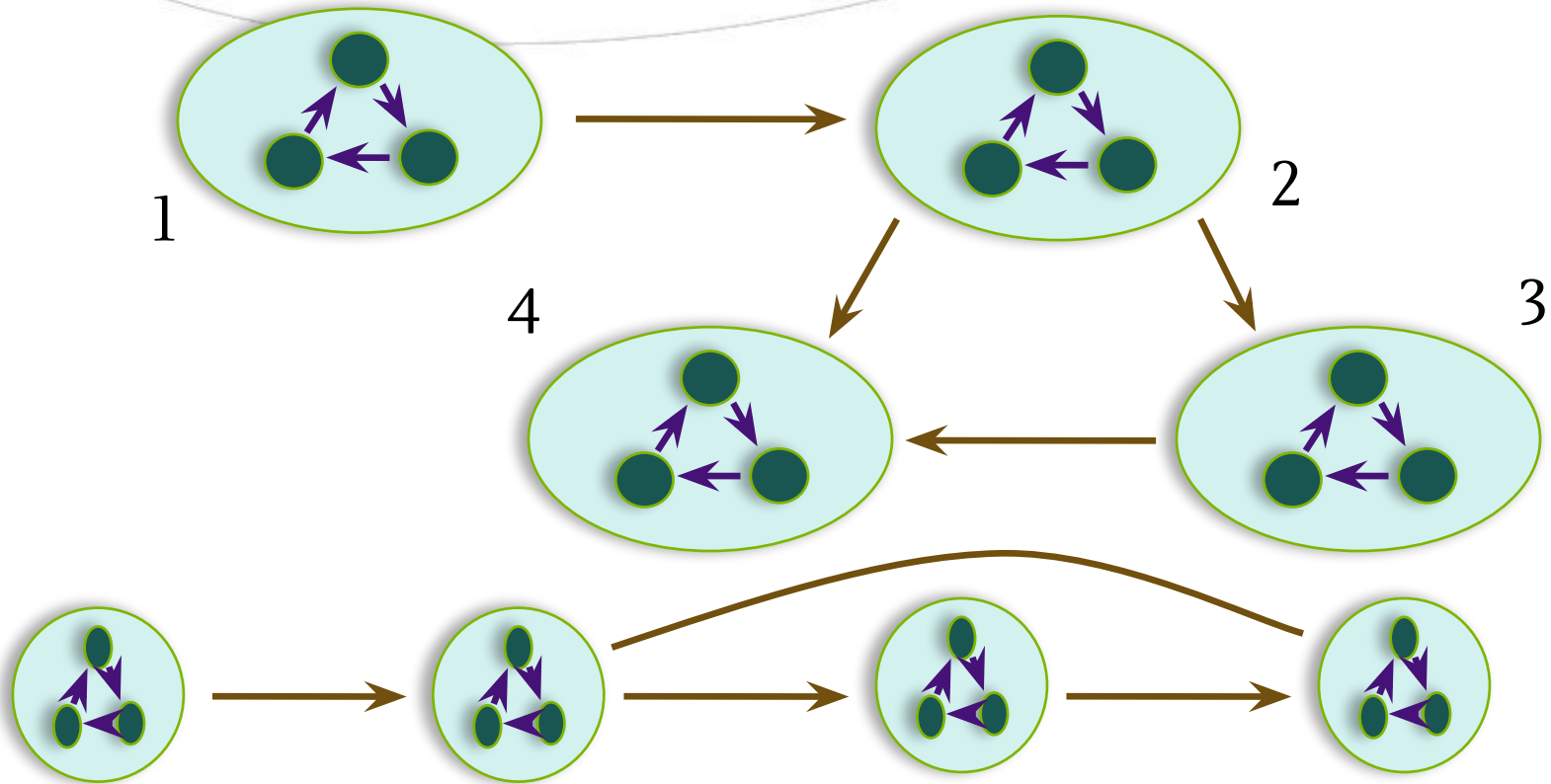


Strongly Connected Component

1. Call DFS(G)
2. Arrange the vertices in order of decreasing $f(u)$



Strongly Connected Component



Strongly Connected Component

